Empowering Development through Data-driven Innovation and Information Technology Solutions

Base IMIS

Deployment Manual

(Windows Environment)

Integrated Municipal Information System (IMIS)

Innovative Solution Pvt. Ltd (ISPL)

Base IMIS Deployment Manual (Windows)

Version 2.0.0

Table of Contents

1. Basic Requirements 1

1.1 Prerequisites 1

1.2 Deployment Engineer Requirements 1

2. Prerequisites Installation 3

2.1 PHP 3

2.1.1 Download PHP 3

2.1.2 Extract the PHP ZIP File 3

2.1.3 Adding the php.ini file in the installed php root folder 4

2.1.4 Add PHP to Windows Environment Variables 4

2.2 Composer 6

2.3 Node.js 6

2.4 PostgreSQL 6

2.5 pgAdmin 8

2.6 Python 9

2.7 Git 10

2.8 Installation Verification Checklist 10

2.9 Clone the Base IMIS Organization 10

2.9.1 Create a destination organization 10

2.9.2 Create a fine-grained GitHub token 11

2.9.3 Install and check the required software 11

2.9.4 Put the script in a folder 12

2.9.5 Clone the organization 12

2.9.6 Check the result and remove the token 12

3. Web Application Setup 13

3.1 Clone the IMIS repository 13

3.2 Create required storage folders 13

3.3 Install PHP dependencies 13

3.4 Install and build JavaScript dependencies 14

3.5 Create the environment file 14

3.6 Generate the Laravel application key 15

4. Database Setup 15

4.1 Importing Database 15

4.2 Configuring Database ENV variables 18

5. Survey Data Import 19

5.1 Preparation of SQL & CSV files for Data Import 19

5.2 Data Import 22

5.2.1 Disable Triggers 22

5.2.2 Import the data 23

5.2.3 Re-enable the triggers 26

6. Build and trigger functions setup 26

7. Configure and encrypt PII data 27

7.1 Create the PII encryption key 27

7.2 Configure PII environment settings 28
7.3 Clear cached configuration 28
7.4 Check the PII configuration 28
7.5 Run the PII database migrations 28
7.6 Seed the PII permission records 29
7.7 Inspect legacy plaintext data 29
7.8 Back up the database 29
7.9 Encrypt existing PII records 29
7.10 Verify all encrypted records 29
7.11 Disable legacy plaintext compatibility 30
7.12 Run the PII test suite 30
8. Configure Users 30
8.1 Configure the super-admin account 30
8.2 Validate and create the user 31
8.3 Verify the user 31
9. Accessing the Web Application 32
10. Installing GeoServer on Windows 32
10.1 Install and verify Java 11 33
10.2 Download the GeoServer Windows installer 37
10.3 Run the GeoServer setup wizard 38
10.4 Check whether GeoServer is running, then start, stop, or restart it 40
10.5 Install the GeoServer extensions 42
10.6 Restart GeoServer safely 44
10.7 Verify that CSS styling is available 44
10.8 Configure and run the Python workspace migration script 45
10.9 Configure the project .env file 49
10.10 CORS issue - GeoServer images or layers are blocked 51

Deploying the Base IMIS (Integrated Municipal Information System) on the Windows environment involves a series of steps and procedures. As the general practice for development and customization is carried out on a windows environment, the following guide provides a detailed walkthrough of the deployment process, including installing the necessary software, configuring the database, and running the IMIS application using Laravel and PostgreSQL.

1. BASIC REQUIREMENTS

The deployment manual is based on the specific system requirements mentioned below and any deviations will require adjustments to the deployment process. The specific system requirements mentioned below are recommended to ensure a smooth and error-free deployment process.

1.1 Prerequisites

The tools mentioned below need to be installed in the system for IMIS deployment. The installation reference is provided in **Section 2**.

- PHP V 8.2.35
- Composer (V 2.11.1)
- Node JS (V 22.23.4)
- PostgreSQL (V 14)
- PostGIS (V3.5)
- GeoServer (V2.28.0)

1.2 Deployment Engineer Requirements

The recommended skillset for the deployment engineer conducting the deployment of IMIS is mentioned below. These skillsets are critical to ensure smooth and error free deployment and maintenance of IMIS.

- PostgreSQL
- PostGIS extension of PostgreSQL
- GeoServer

Apart from the deployment engineer, the minimum recommended skillset for the development/ maintenance team is mentioned below:

- PHP language
- Laravel Framework
- PostgreSQL
- PostGIS extension of PostgreSQL
- NPM package manager
- Git version control
- GeoServer
- JavaScript
- jQuery
- Open Layers (OL)
- Technical Documentation

2. PREREQUISITES INSTALLATION

PHP 8.2 | Composer | Node.js | PostgreSQL | pgAdmin | Python | Git

2.1 PHP

2.1.1 Download PHP

i. Open a web browser, such as Google Chrome or Microsoft Edge.
ii. Open the official direct download link shown below.

https://downloads.php.net/~windows/releases/archives/php-8.2.35-1-nts-Win32-vs16-x64.zip

iii. Wait for the ZIP file to finish downloading.
iv. Confirm that the downloaded file is named as shown below.

```
php-8.2.35-1-nts-Win32-vs16-x64.zip
```

Use the following PHP package selections:

Setting	Required selection
PHP version	PHP 8.2.35
Architecture	x64
Thread safety	Non Thread Safe (NTS)
Compiler	VS16
Package	Zip



Figure 1. PHP package selection / release reference

2.1.2 Extract the PHP ZIP File

Open the folder where the browser saved the downloaded ZIP file.
Locate the PHP ZIP file.

```
php-8.2.35-1-nts-Win32-vs16-x64.zip
```

Right-click the ZIP file and select Extract All.

Rename the folder to “php8.2.35” and extract to the Root folder of C:/”

The remainder of this guide represents that folder as:

```
<YOUR_PHP_INSTALL_PATH>
```

PHP can be installed on any local drive. Example paths:

```
C:\php8.2.35
D:\php8.2.35
```

After extraction, confirm that the PHP folder contains:

```
php.exe
php.ini-development
php.ini-production
ext
```

2.1.3 Adding the php.ini file in the installed php root folder

- Link or the file “php.ini”
- Copy and paste the php.ini file in the root folder of “php8.2.35/”

2.1.4 Add PHP to Windows Environment Variables

The PHP folder must be added to the Windows Path variable so commands such as php -v can be executed without specifying the full path to php.exe.

i. Click the Windows Start button or press the Windows key.
ii. ***Search for*** “environment variables”.
iii. Click Edit environment variables for your account.
iv. Select the “Environment Variables”, and under the “System Variables” section, select the “Path” and click on “Edit”.
v. In the dialog box, select “New” and Paste the copied path (“***C:\php-8.2.35***”) in the new line.
vi. Save the changes and open a new Command Prompt or PowerShell window.

```
php -v
Use this command to verify that PHP is available from the command line.
The output should show PHP 8.2.35 (cli) (built: Oct 26 2026 09:48:31) (NTS Visual C++ 2019 x64)
```

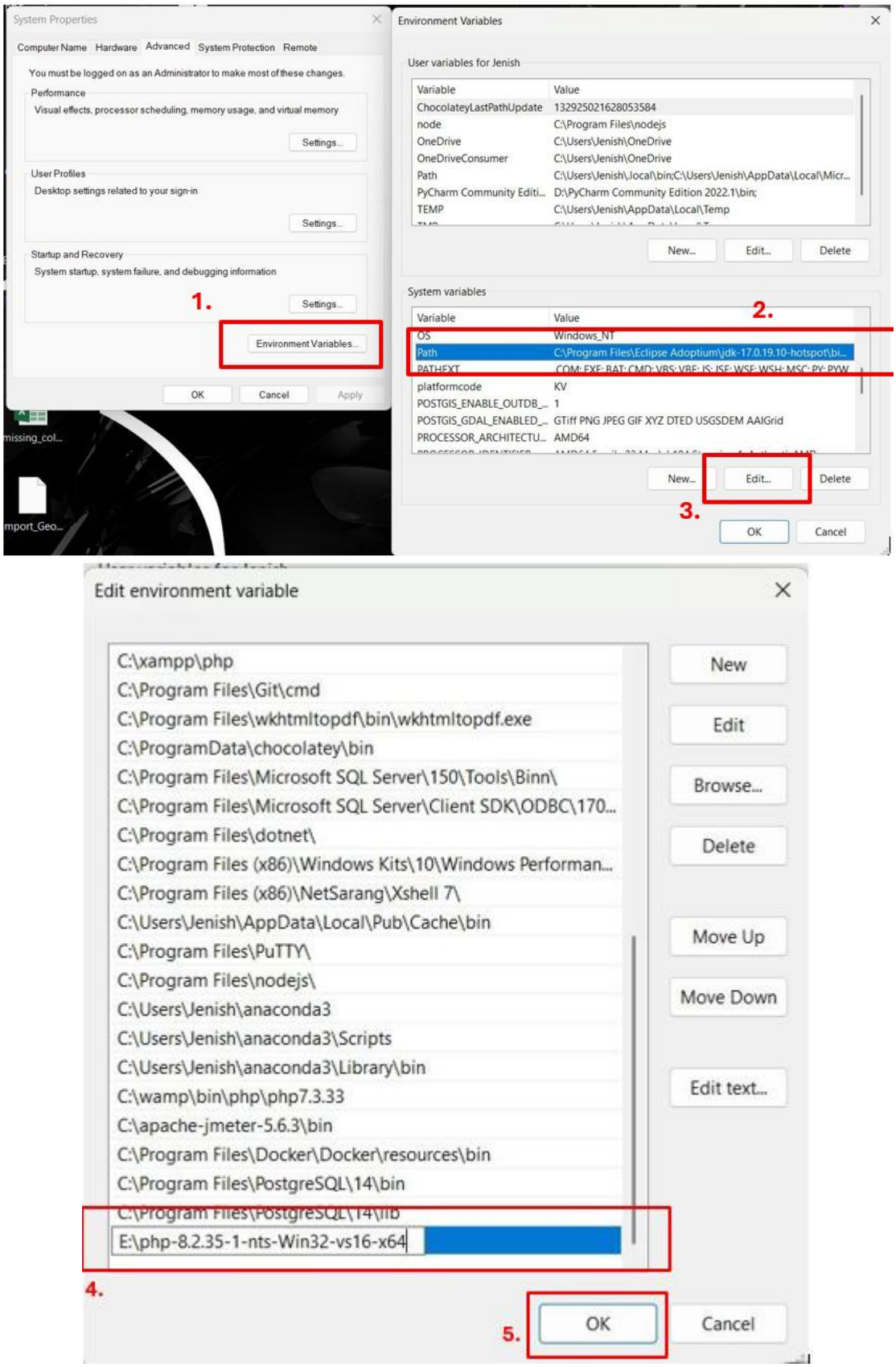


Figure 2. *Windows Environment Variables / Path configuration*

2.2 Composer

Installer: https://getcomposer.org/Composer-Setup.exe

i. Download and open Composer-Setup.exe.
ii. Leave Developer mode unchecked and select the PHP executable from the PHP installation directory.

```
<YOUR_PHP_INSTALL_PATH>\php.exe
```

Example:

```
C:\php8.2.35
```

iii. Leave the proxy option unchecked, click Next, and then click Install.
iv. After installation, click Finish and open a new Command Prompt.
v. Verify Composer using the command below in a new terminal.

```
composer --version
```

The output should show the Composer version 2.11.1 2026-10-30 14:52:10 and PHP 8.2.35.

Configuration reminder: Confirm the PHP ini file and Windows environment variable configuration before troubleshooting Composer.

2.3 Node.js

Windows x64 installer: https://nodejs.org/dist/v22.23.4/node-v22.23.4-x64.msi

i. Download the Node.js 22.23.4 x64 Windows installer.
ii. Open the downloaded installer and click Next.
iii. Accept the licence agreement and keep the default installation location.
iv. Keep the following components selected:
v. Node.js runtime
vi. npm package manager
vii. Add to PATH
viii. Click Next, then Install and Finish.
ix. Open a new Command Prompt and verify the installation.

```
node --version
npm --version
The output should show node v22.23.4 and npm version 10.9.9
```

2.4 PostgreSQL

Required version: **14.x**

i. Open the official PostgreSQL Windows download page.
https://sbp.enterprisedb.com/getfile.jsp?fileid=1260635

ii. Click Download the installer, select PostgreSQL 14.24 for Windows x86-64, and download the installer.

iii. Open the installer and keep the following components selected:

   a. PostgreSQL Server
   b. pgAdmin 4
   c. Stack Builder
   d. Command Line Tools

iv. Keep the default installation and data folders.

v. Create and safely record a password for the postgres database user.

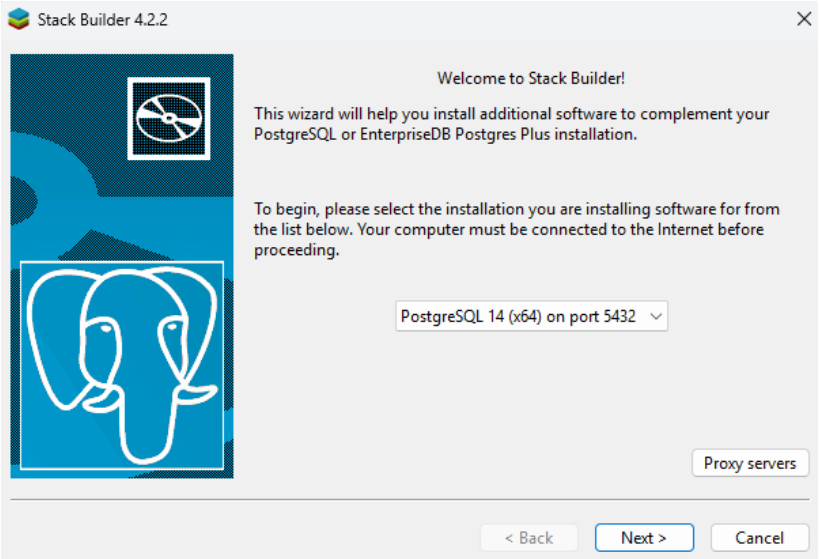
vi. Keep the default port: 5432. If the port is already occupied, assign the next available port (eg: 5433,5434…)

vii. Keep the default locale, click Next, and then click Install.

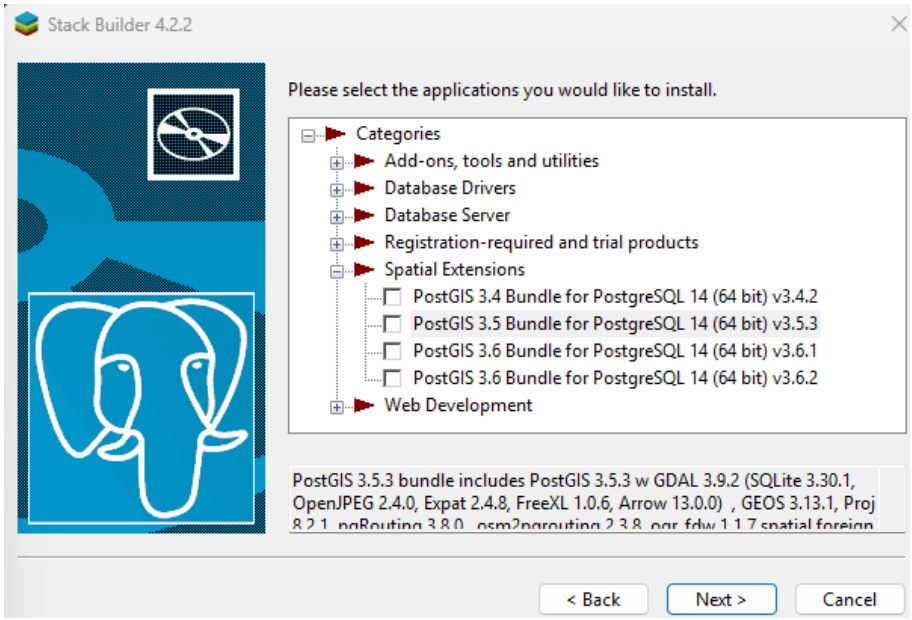
viii. When installation finishes, click Finish.

ix. Stack Builder should open; if it doesn't search for "Application Stack Builder" and open it.

x. Select installed PostgreSQL (x64) on port 5432



xi. Expand the Spatial Extensions and select PostGIS 3.5 bundle and click next (note: installation may take time and freeze, please wait for it to respond)



xii. Continue installation with the default values.

xiii. Add the installation path in the Environment Variables similar to the above steps.

   a. C:\Program Files\PostgreSQL\14\bin

   b. C:\Program Files\PostgreSQL\14\lib

xiv. Open command prompt and verify postgres

```
psql --version
```

The output should display psql (PostgreSQL) 14.24

2.5 pgAdmin

Download page:

https://ftp.postgresql.org/pub/pgadmin/pgadmin4/v9.19/windows/pgadmin4-9.19-x64.exe

i. Download and open the latest Windows installer.

ii. Keep the default installation options and click Install.

iii. After installation, click Finish and open pgAdmin 4.

iv. Create a pgAdmin master password when requested.

   a. If the installed postgres version is already installed, you need to add the password manually.

v. Right Click on Servers and Register->Server

vi. Enter the connection settings below.

Setting	Value
General -> Name	PostgreSQL 14

Connection -> Host	localhost
Connection -> Port	5432
Maintenance database	postgres
Username / Password	postgres / <POSTGRES_PASSWORD>

vii. Click Save.

viii. Confirm that the PostgreSQL 14 server appears in pgAdmin.

Note: Installing pgAdmin separately does not reinstall PostgreSQL. pgAdmin only connects to and manages the existing PostgreSQL server.

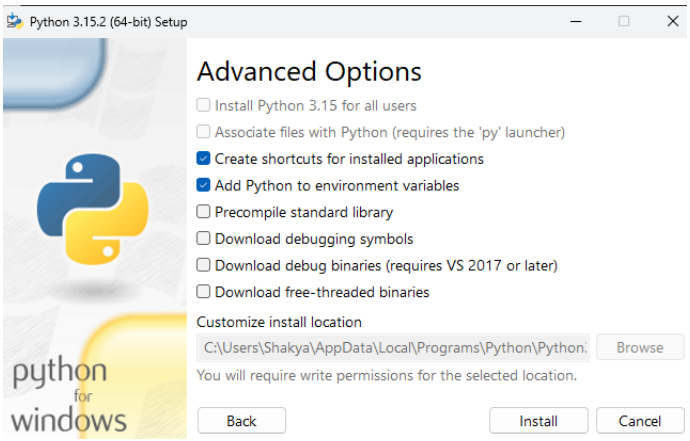
2.6 Python

Installer link provided in the source document:
https://www.python.org/ftp/python/3.15.2/python-3.15.2-amd64.exe

i. Open the official Python release page indicated in the source guide (Download Python 3.13.17).

ii. Download the Windows installer (64-bit).

iii. Open the installer and select Add python.exe to PATH.



iv. Click Install Now.

```
python --version
pip --version
```

The source guide expects Python 3.13.17 and the installed pip version to be displayed.

v. When installation finishes, click Disable path length limit, then click Close.

vi. Open a new Command Prompt and verify Python and pip.

2.7 Git

Download Git form official website: git-scm.com/install/windows

i. Run the installer with default options.
ii. Finish the installation
iii. Check if Git is installed using cmd.

```
git --version
```
The output should display git version x.x

2.8 Installation Verification Checklist

After completing the installations, open a new Command Prompt and verify the command-line tools:

```
php -v
composer --version
node --version
npm --version
psql --version
python --version
pip --version
git –version
```

2.9 Clone the Base IMIS Organization

It is recommended to clone the entire Base IMIS open-source organization while working for a new deployment to enable a centralized management of the source code, technical documentation and other necessary documentation. Managing all required documentation in one organization allows easy knowledge management and transfer.

2.9.1 Create a destination organization

i. Sign in to GitHub.
ii. On the GitHub landing page, select the + button in the upper-right corner.
iii. Choose New organization.
iv. Select the Free plan, complete the requested organization details, and choose an available organization name, such as imis-training-yourname.
v. Complete the setup. If prompted to invite members, skip this for the exercise.
vi. Open the new organization and confirm its name. Leave it empty; the script will create repositories there.

vii. Use an organization you own or where you have permission to create repositories. Do not use base-imis as the destination. The script creates public repositories, so use an organization approved for public training repositories.

2.9.2 Create a fine-grained GitHub token

i. Open your GitHub profile menu, choose Settings, then Developer settings.

ii. Under Personal access tokens, choose Fine-grained tokens, then Generate new token.

iii. Name the token (for example, IMIS organization-copy training) and set an expiry that covers the exercise.

iv. Set Resource owner to your destination organization that you have created. Set Repository access to All repositories.

v. Grant the permissions below, then generate and copy the token. GitHub may require an organization owner to approve it.

Permission	Access	Why it is needed
Administration	Read and write	Create repositories and set each copied repository's default branch.
Contents	Read and write	Upload the copied Git history, branches and tags.
Metadata	Read-only	Required repository information; GitHub normally includes this automatically.

Your GitHub account must also be allowed to create and administer repositories in the destination organization. The token cannot grant permissions your account does not have. Keep the token private; the script asks for it using hidden input. Copy the token.

2.9.3 Install and check the required software

Install Python 3.11.9 from the official Python Windows installer. During setup, select Add python.exe to PATH. Install Git for Windows using its standard setup options. Close and reopen the terminal after installing either program.

Open Command Prompt or PowerShell and check the versions:

```
python --version
python -m pip --version
git --version
```

Python should report 3.15.2. Install the required GitHub library:

```
python -m pip install PyGithub==2.5.0
```

If Python is not found or the version differs, ask the trainer to help select Python 3.11.9 before continuing.

2.9.4 Put the script in a folder

Save the provided Python file (Location of python file: Deployment_documentation->Clone_organization.py as Clone_organization.py in a folder you can find, for example C:\IMIS-Training. No PowerShell setup file is required. Open a terminal in that folder, or change to it:

```
cd C:\IMIS-Training
```

2.9.5 Clone the organization

Open the terminal in the folder and run the script:

```
python Clone_organization.py
```

Enter the destination organization name only when prompted, not the full GitHub address. Paste the token at the hidden prompt and press Enter; the characters will not appear. Review the listed source repositories and target. Type the target organization name exactly at the confirmation prompt to proceed. Press Enter without typing it to cancel.

2.9.6 Check the result and remove the token

The script reports each repository as COPIED, SKIPPED or FAILED. Check the destination organization on GitHub and confirm the repositories, files and branches are present. Existing destination repositories are skipped. If a repository reports FAILED after creation, inspect it before trying again; the script will skip it on a rerun.

When finished, delete the temporary token from GitHub Settings → Developer settings → Fine-grained tokens.

What is copied

The script copies public repositories' Git history, branches and tags. It does not copy issues, pull requests, releases or attachments, repository settings, Git LFS objects or separately hosted submodule repositories.

If a copy reports HTTP 403

Check that the token belongs to the destination organization, is approved and includes Administration: Read and write and Contents: Read and write. Confirm your GitHub account has permission to create and administer repositories in that organization. Do not delete or rerun over a repository until you have checked whether files were uploaded.

3. WEB APPLICATION SETUP

The following steps are recommended to be carried out to ensure deployment of IMIS in windows environment.

3.1 Clone the IMIS repository

The first step is to clone the **web_app.git** repository that contains the IMIS web application source code. Choose a permanent local folder for the IMIS source code. Open Command Prompt or PowerShell in that folder and run:

git clone <GIT_REPOSITORY_URL>

Replace **<GIT_REPOSITORY_URL>** with the repository of the web_app repository of the organization. After cloning, change to the project root:

cd <IMIS_PROJECT_FOLDER>

All commands in the remaining sections must be run from the folder containing artisan, composer.json and package.json.

3.2 Create required storage folders

Create the folders used for building-survey KML files, house images and emptying receipts:

```
mkdir storage\app\public\building-survey-kml
mkdir storage\app\public\emptyings\houses
mkdir storage\app\public\emptyings\receipts
```

Confirm that the following folder structure exists:

```
storage\app\public\building-survey-kml
storage\app\public\emptyings\houses
storage\app\public\emptyings\receipts
```

3.3 Install PHP dependencies

From the IMIS project root, run:

```
composer install
```

If this command reports a security-blocking error, run:

```
composer update --no-security-blocking
```

Wait for Composer to finish installing the project dependencies before continuing.

After the composer is successfully installed, run the following command from the IMIS project root:

```
php artisan storage:link
```

This command creates a symbolic link from `public/storage` to `storage/app/public`, allowing files stored by the application to be accessed through the web browser.

3.4 Install and build JavaScript dependencies

Install the packages defined in package.json:

npm install

Build the development assets:

npm run dev

The command compiles the application's JavaScript and CSS assets. Keep the terminal open while the development process is running.

Resolve the Swiper export

During npm run dev, the following error may appear:

Module not found: Error: Package path ./swiper-bundle.min.js is not exported from package <PROJECT_FOLDER>/node_modules/swiper

If this error occurs:

- Open node_modules\swiper\package.json.
- Locate the "exports" object.
- Add the following export entry, ensuring the surrounding JSON commas remain valid:
  "./swiper-bundle.min.js": "./swiper-bundle.min.js",
- Save the file, then run the build again:
  npm run dev

This change is inside node_modules and may be overwritten by a future npm install. If the project repeatedly requires this workaround, record the compatible Swiper version or apply a maintained project-level fix.

3.5 Create the environment file

Laravel requires a local .env file. If it does not already exist, create it from the project template:

copy .env.example .env

Do not overwrite an existing configured .env file. Database and service-specific values will be configured in their corresponding deployment sections.

3.6 Generate the Laravel application key

Run the following command from the project root:

php artisan key:generate

The expected confirmation is:

Application key set successfully.

The command writes a generated APP_KEY value to the local .env file. Do not copy an application key from another deployment.

4. DATABASE SETUP

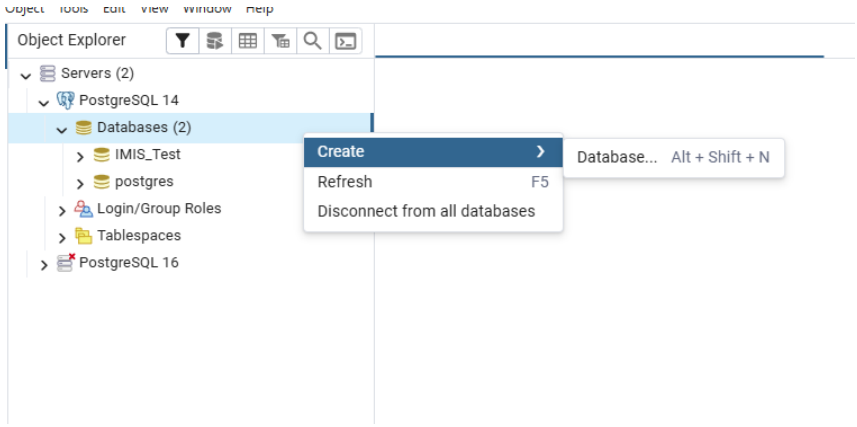
These steps imports the blank database and configures the necessary environment variables to enable the connection of the database with the web application.

4.1 Importing Database

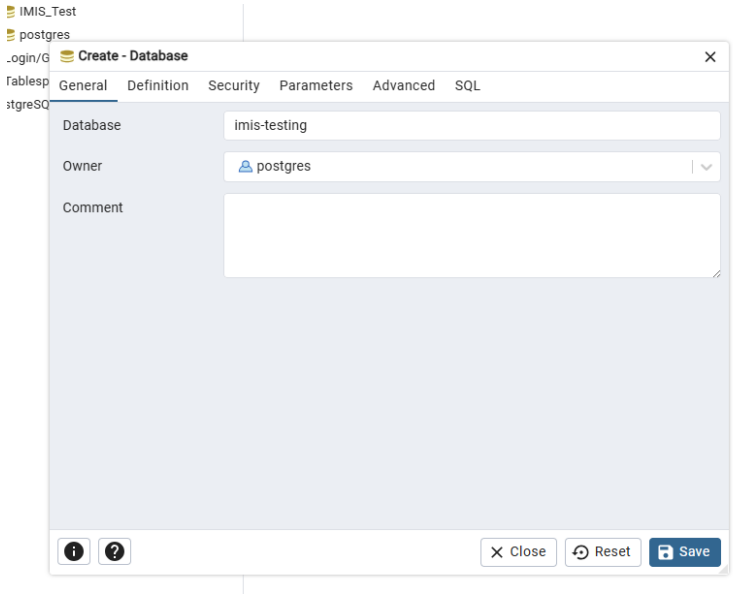
i. Locate the database SQL file in the GitHub organization (Deployment_documentation->database)
ii. Download one of the provided SQL file

- imis_db_with_data.sql: IMIS Database with complete dummy data
- imis_db_with_lookup.sql: IMIS Database with the minimum required datasets (look up values, reference values, standard values, and other tables that are required for proper functioning of IMIS)

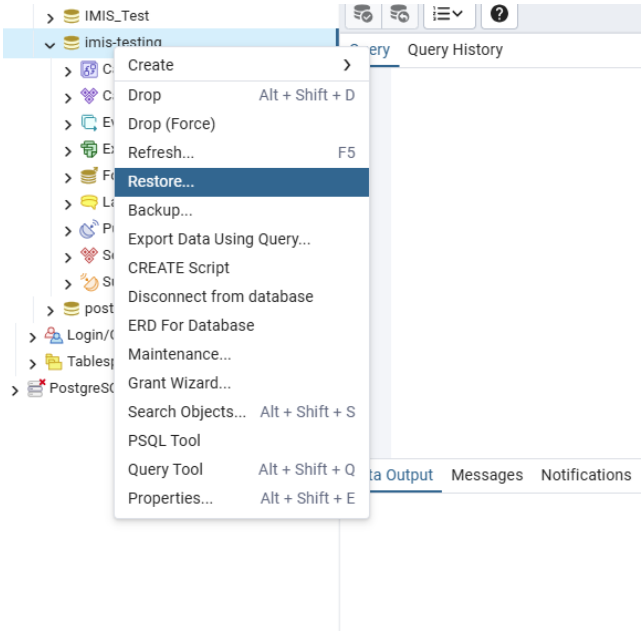
iii. Save the downloaded file in an accessible local folder.
iv. Open pgAdmin and connect to the local PostgreSQL server.
v. In the left navigation panel, right-click Databases and select Create > Database.



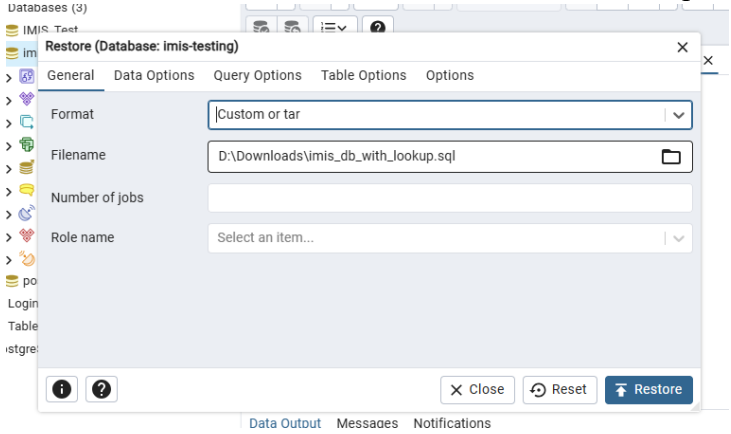
vi. Enter the deployment database name, select the correct owner, and click Save.

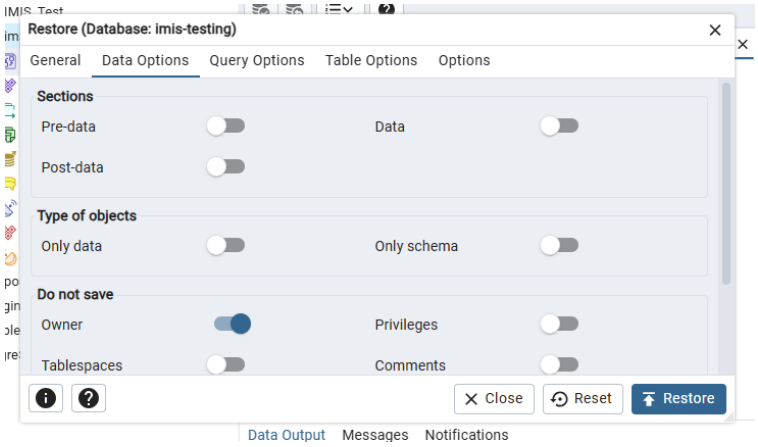


vii. Right Click the created database and click “Restore”.

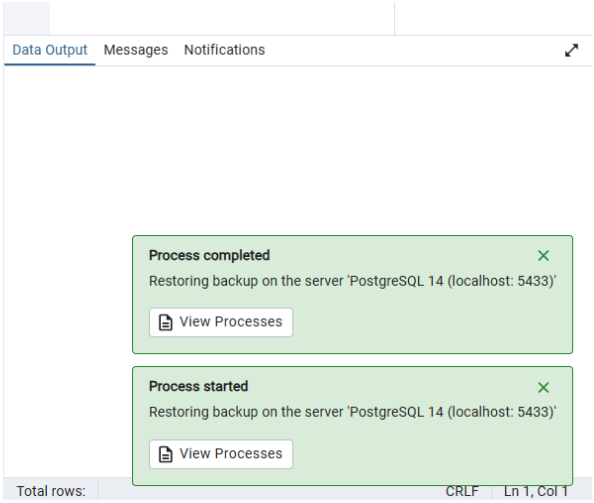


viii. Choose the downloaded SQL file in the General Tab. Also, in the Data Options, check the “Owner” in the Do not save section and Restore the sql file.

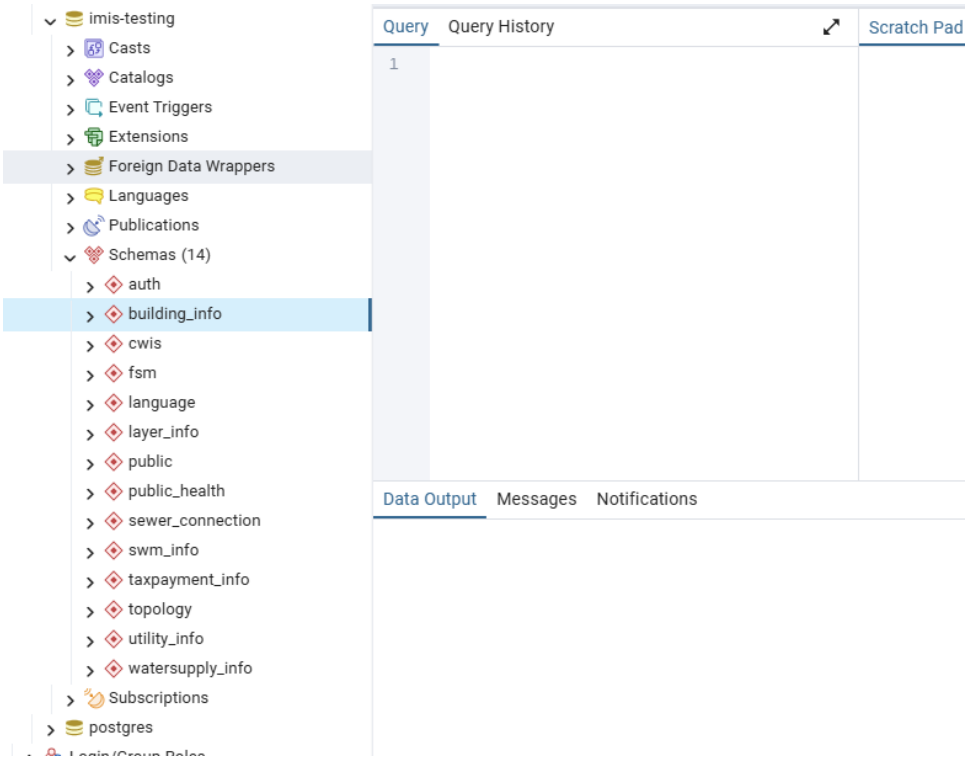




ix. Check the Messages panel and wait for the backup restore to finish without an error.



x. Refresh the database and confirm that the required schemas, tables, lookup data, triggers, and database functions are present.



4.2 Configuring Database ENV variables

Open the project's .env file and configure the PostgreSQL connection:

DB_CONNECTION=pgsql

DB_HOST=127.0.0.1

DB_PORT=5432

DB_DATABASE=<DB_DATABASE>

DB_USERNAME=<DB_USERNAME>

DB_PASSWORD=<DB_PASSWORD>

Use the actual local database name and credentials. Change DB_PORT only if PostgreSQL was configured on a port other than 5432. Do not commit the local .env file or its passwords to Git.

Before continuing, clear the cached configuration:

php artisan config:clear

5. SURVEY DATA IMPORT

This section outlines the steps required to import survey data created through the census survey and processed into a structured GPKG file, following the structure of the data dictionary.

The IMIS application requires the initial spatial and non-spatial datasets collected from the field surveys to be imported into the database prior to deployment. It is recommended to import the data locally first, and then restore the instance of that database into the production server. The sequence for the data import provided in the Table 1 must be strictly followed to ensure error-free import of data, as altering the sequence may lead to mismatched foreign keys of the relational tables.

5.1 Preparation of SQL & CSV files for Data Import

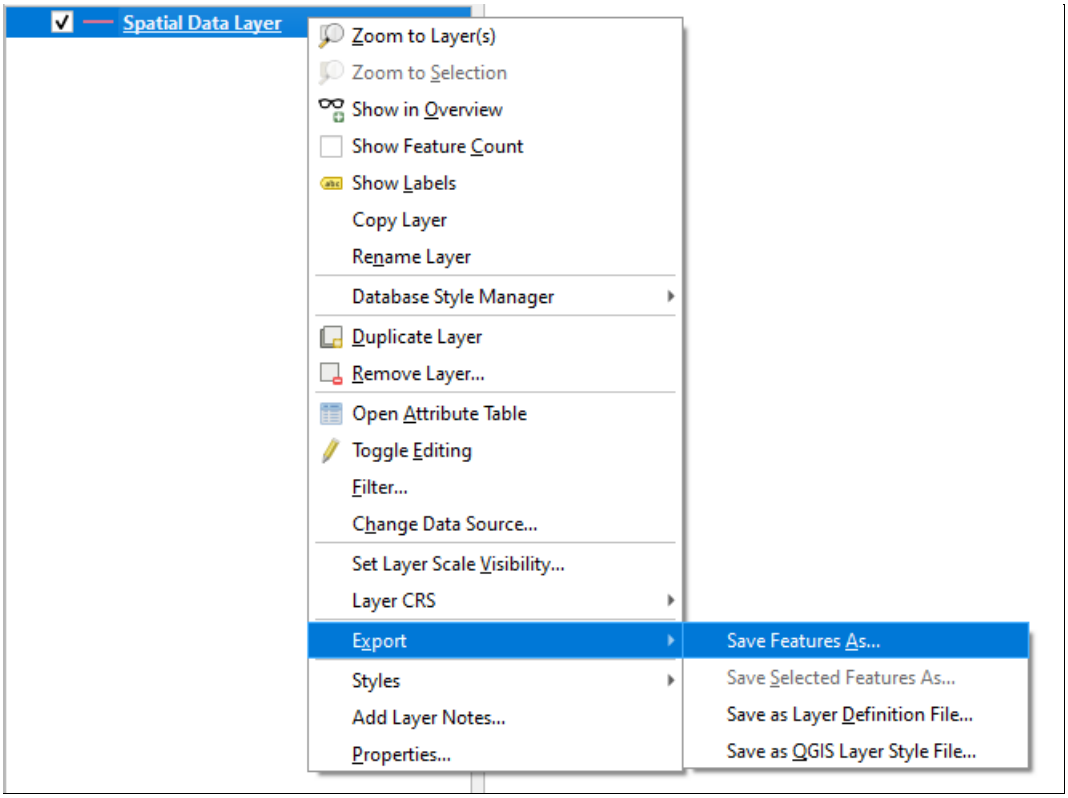
The recommended methodology to import the spatial and attribute data into the database is provided below:

Step 1 : Data Preparation: Ensure the survey data layers are prepared in a GeoPackage in QGIS with the proper database structure (table names, column names, foreign keys, etc.) as mentioned in the Data Dictionary of IMIS.

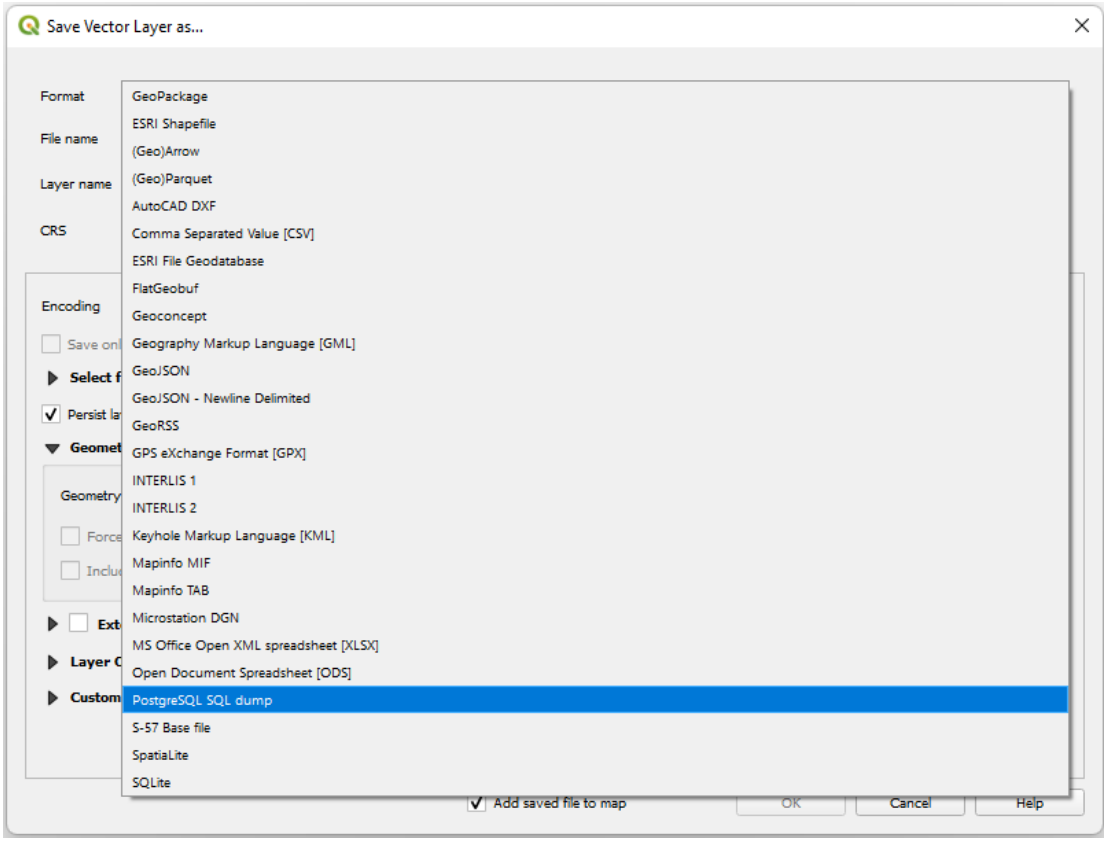
Step 2 : Creating PostgreSQL SQL dump file for each of the individual tables mentioned in Table 1.

a. Select the 'Spatial data Layer' with all the required attributes (following the data dictionary)

b. Right click and go to 'Export' option and select 'Save Feature As…'



c. Select the format: PostgreSQL SQL Dump



d. Select File Location and Give filename same as Table Name.

e. Example: for table roads:



f. In Layer Options:

i. CREATE_SCHEMA: NO

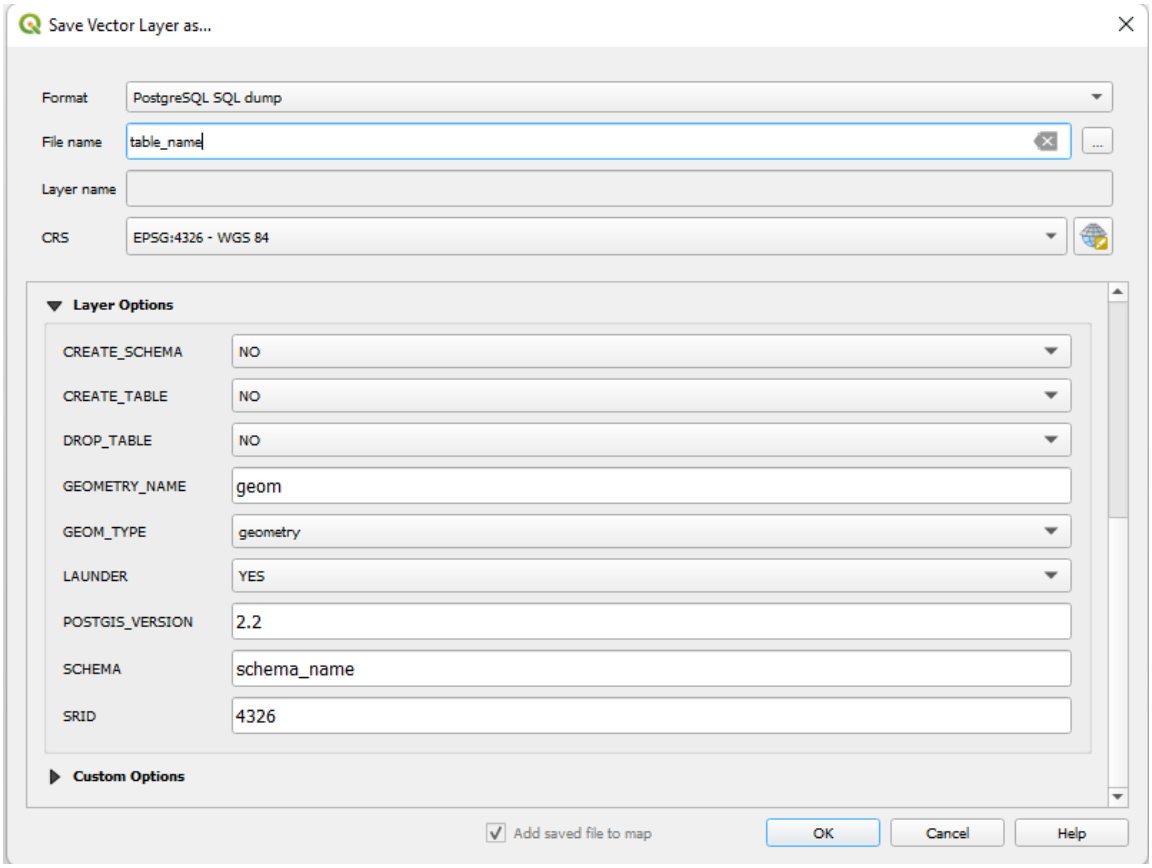
ii. CREATE_TABLE: NO

iii. DROP_TABLE: NO

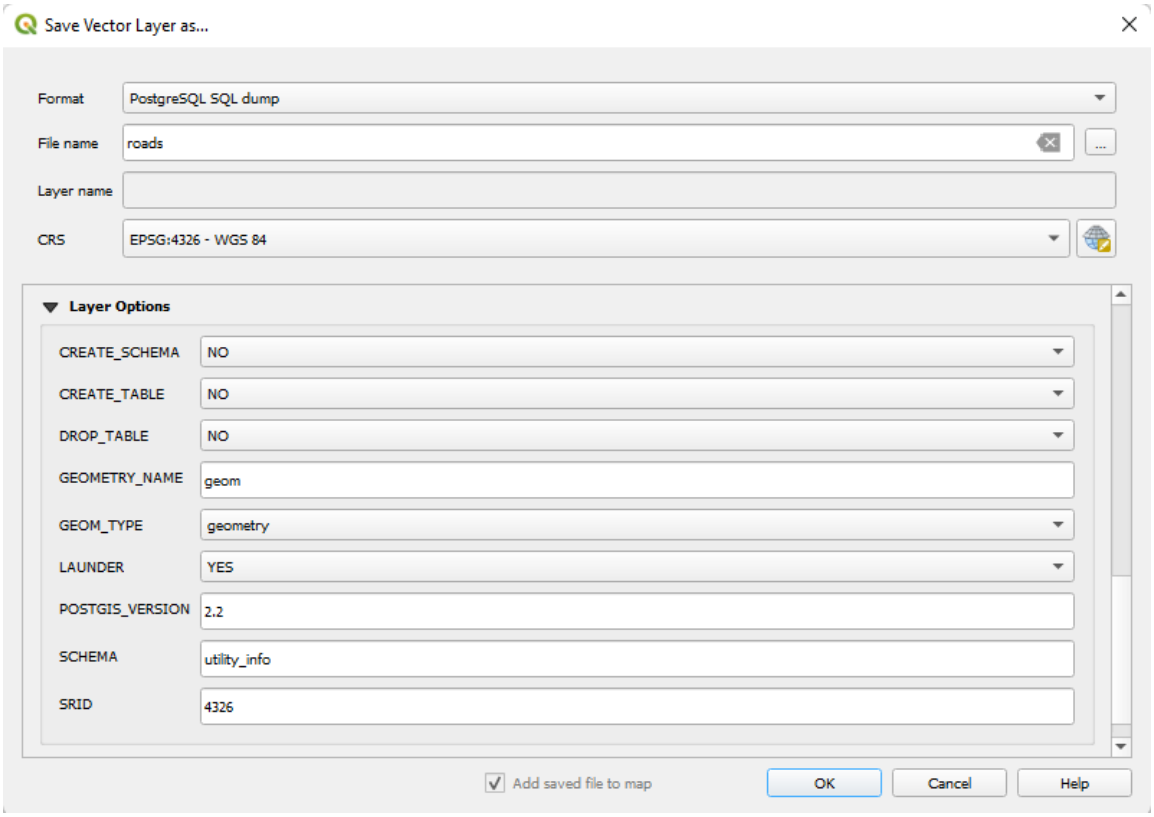
iv. GEOMETRY_TYPE: GEOM

v. SCHEMA: SCHEMA_NAME

vi. SRID: 4326



g. In Layer Options: For Table ‘roads’



Step 3 : Once all SQL files are prepared, prepare the CSV files for the non-spatial data. The import sequence should be followed.

5.2 Data Import

While new cities will have new datasets, for the purposes of demonstration, a structured dataset required for importing the dummy dataset is provided in the GitHub organization's deployment_documentation repository under the sample_import_data folder. Download it if necessary to test the data import process

5.2.1 Disable Triggers

i. Open pgAdmin and connect to the PostgreSQL server.

ii. Select the IMIS database.

iii. Right-click the database and select Query Tool.

iv. Enter and execute the following commands:

```
ALTER TABLE building_info.buildings DISABLE TRIGGER USER;
ALTER TABLE utility_info.roads DISABLE TRIGGER USER;
ALTER TABLE fsm.containments DISABLE TRIGGER USER;
ALTER TABLE cwis.data_cwis DISABLE TRIGGER USER;
```

v. Confirm that all four commands finish without errors.

5.2.2 Import the data

The data needs to be imported in a certain sequence provided in the table below. There are two types of data to be imported i.e., spatial data and non-spatial data. Spatial data is provided in .SQL and non-spatial data is provided in .CSV format. Based on the extension of the data, either Step A or Step B should be followed.

a) Importing SQL Data

i. Open pgAdmin and connect to the PostgreSQL server.

ii. Select the IMIS database.

iii. Right-click the database and select Query Tool.

iv. Click Open File on the Query Tool toolbar.

v. Select the next .sql file in the sequence shown above.

vi. Confirm that the correct SQL script appears in Query Tool. Alternatively, the SQL file can be drag and dropped into the Query Tool.

vii. Click Execute Script or press F5.

viii. Check the Messages panel and confirm that the script completed without an error.

ix. Refresh the target schema and table before continuing to the next numbered file.

b) Importing CSV Data

The destination tables must already exist. Import each CSV file into its matching table:

- 18.build_contains.csv -> building_info.build_contains
- 19.owners.csv -> building_info.owners
- 21.build_toilets.csv -> fsm.build_toilets

i. In pgAdmin, expand Servers > PostgreSQL > Databases > IMIS database > Schemas.

ii. Expand the required schema, expand Tables, right-click the destination table, and select Import/Export Data.

- Schema-> building_info-> Tables->build_contains
- Schema-> building_info-> Tables->owners
- Schema-> fsm ->Tables-> build_toilets

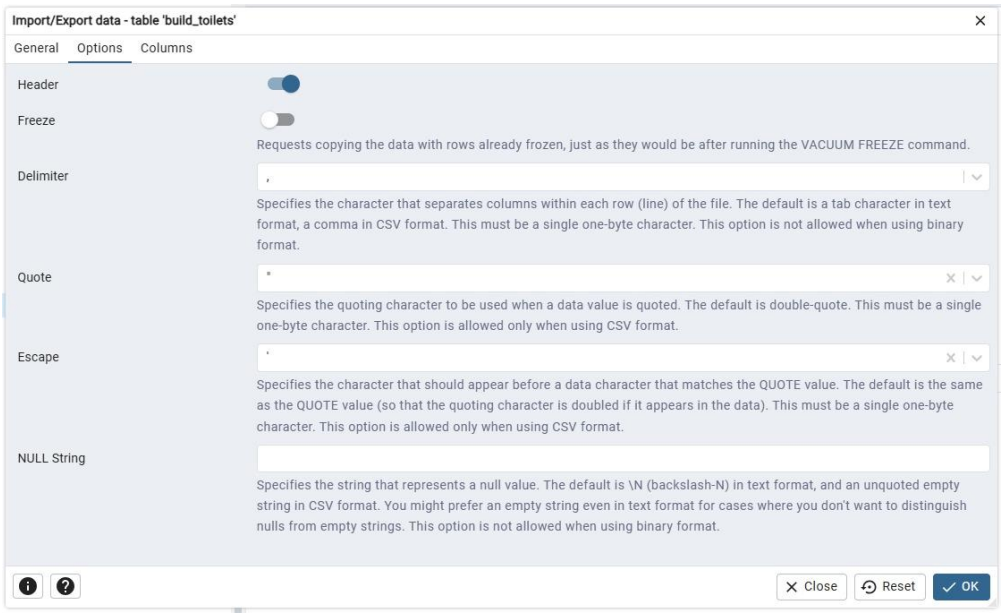
iii. On the General tab, select Import and choose the required CSV file. Do not import it without completing the following steps.

iv. Set Format to csv and Encoding to UTF8.

v. On the Options tab, turn Header on.

vi. Set Delimiter to a comma (,), Quote to a double quotation mark ("), and Escape to a double quotation mark (").

vii. Open the Columns tab and select the columns in the same order in which they appear in the CSV file.



viii. Click OK to begin the import.

ix. Confirm that pgAdmin reports a successful import.

x. Open View/Edit Data > All Rows and confirm that the imported records are present.

xi. Continue with the next numbered file only after the current import finishes without errors.

Do not import additional SQL or CSV files unless the deployment owner provides an approved sequence.

Import Sequence

S. N.	Schema	Database table	SQL filename	Type
1	`layer_info`	`layer_info.citypolys`	`1.citypolys.sql`	Spatial

S. N.	Schema	Database table	SQL filename	Type
2	layer_info	layer_info.landuses	2.landuses.sql	Spatial
3	layer_info	layer_info.waterbodys	3.waterbodys.sql	Spatial
4	layer_info	layer_info.wardboundary	4.wardboundary.sql	Spatial
5	layer_info	layer_info.grids	5.grids.sql	Spatial
6	layer_info	layer_info.wards	6.wards.sql	Spatial
7	layer_info	layer_info.ward_overlay	7.ward_overlay.sql	Spatial
8	layer_info	layer_info.sanitation_system	8.sanitation_system.sql	Spatial
9	utility_info	utility_info.roads	9.roads.sql	Spatial
10	utility_info	utility_info.water_supplys	10.water_supplys.sql	Spatial
11	fsm	fsm.treatment_plants	11.treatment_plants.sql	Spatial
12	utility_info	utility_info.sewers	12.sewers.sql	Spatial
13	utility_info	utility_info.drains	13.drains.sql	Spatial
14	layer_info	layer_info.places	14.places.sql	Spatial
15	layer_info	layer_info.low_income_communities	15.low_income_communities.sql	Spatial
16	building_info	building_info.buildings	16.buildings.sql	Spatial
17	fsm	fsm.containments	17.containments.sql	Spatial
18	building_info	building_info.build_contains	18.build_contains.csv	Non-spatial
19	building_info	building_info.owners	19.owners.csv	Non-spatial
20	fsm	fsm.toilets	20.toilets.sql	Spatial
21	fsm	fsm.build_toilets	21.build_toilets.csv	Non-spatial

5.2.3 Re-enable the triggers

After every SQL and CSV file has been imported successfully, return to Query Tool and execute:

```
ALTER TABLE building_info.buildings ENABLE TRIGGER USER;
ALTER TABLE utility_info.roads ENABLE TRIGGER USER;
ALTER TABLE fsm.containments ENABLE TRIGGER USER;
ALTER TABLE cwis.data_cwis ENABLE TRIGGER USER;
```

Confirm that all four commands finish successfully before continuing.

6. BUILD AND TRIGGER FUNCTIONS SETUP

After completing the data import and re-enabling the triggers in pgAdmin, switch from pgAdmin to a terminal opened in the IMIS project folder.

i. Open the IMIS project folder in File Explorer.

ii. Click the File Explorer address bar, type powershell, and press Enter. This opens PowerShell in the project folder.

iii. Alternatively, open PowerShell and change to the project folder using the command below.

cd "<IMIS_PROJECT_FOLDER>"

iv. Confirm that the folder contains the artisan file by running:

dir artisan

v. Run each command in the table below individually. Wait for a success message before running the next command.

Do not run these commands in pgAdmin Query Tool. They are Laravel Artisan commands and must be run from the IMIS project root.

Purpose	**Command**
Create functions and triggers that update grid and ward counts	`php artisan buildfunction:updatecount`
Create or replace the database functions used by map tools	`php artisan buildfunction:maptool`
Create functions used after importing tax-payment data	`php artisan buildfunction:tax`
Create functions used after importing water-supply payment data	`php artisan buildfunction:watersupply`

Purpose	Command
Create functions used after importing solid-waste payment data	`php artisan buildfunction:swmpayment`
Create quarter records used by the FSM KPI dashboards	`php artisan kpi:cron`
Install the function that generates UUID public IDs	`php artisan public-id:install-function`
Generate UUID public IDs for building BIN and containment records	`php artisan public-id:generate`

7. CONFIGURE AND ENCRYPT PII DATA

7.1 Create the PII encryption key

Create a secure folder outside the project repository:

```
New-Item -ItemType Directory -Force "C:\secure-keys"
```

Generate a random 32-byte PII key and save it as Base64 text:

```
php -r "file_put_contents('C:/secure-keys/base-imis-pii-v1.key',
base64_encode(random_bytes(32)));"
```

Security requirements:

- Generate the key only once for each database/environment.
- Keep the key outside the Git repository.
- Never commit the key or include it in `.env`.
- Never send the key through chat or ordinary email.
- Building, Application, and Property Tax PII use the same active PII key.
- Back up the key securely because encrypted records cannot be recovered without it.

Existing encrypted database warning

If the developer receives a copy of a database that already contains encrypted PII, do not generate a different key. The developer must securely receive the matching key from the authorized administrator.

A newly generated key cannot decrypt data encrypted using another key.

7.2 Configure PII environment settings

Add or update the following values in `.env`:

```
PII_KEY_PROVIDER=file
PII_ACTIVE_KEY_VERSION=v1
PII_KEY_V1_FILE=C:/secure-keys/base-imis-pii-v1.key
PII_KEY_V2_FILE=
PII_ALLOW_LEGACY_PLAINTEXT=true
PII_AUDIT_ENABLED=true
PII_KEY_BACKUP_INTERVAL_DAYS=60
PII_KEY_BACKUP_WARNING_DAYS=5
PII_EXPORT_MAX_BINS=5000
PII_EXPORT_MAX_APPLICATION_IDS=5000
PII_EXPORT_MAX_TAX_CODES=5000
PII_EXPORT_MAX_FILE_KB=2048
```

Using forward slashes in the Windows key-file path avoids path parsing problems.

7.3 Clear cached configuration

Run:

```
php artisan optimize:clear
```

Run this again whenever PII values in `.env` are changed.

7.4 Check the PII configuration

Run:

```
php artisan pii:check-configuration
```

Expected information includes:

```
Provider: file
Active key version: v1
Cipher: aes-256-gcm
PII configuration check passed.
```

Do not continue with data encryption if this command fails.

7.5 Run the PII database migrations

The supplied IMIS database already contains the Base-IMIS tables. From the IMIS project terminal, run:

```
php artisan migrate:pii
```

This command runs the migrations required by the PII feature in the correct dependency order. Laravel records each completed migration in the database.

7.6 Seed the PII permission records

Run:

```
php artisan db:seed --
class="Database\Seeders\OwnerPiiPermissionsSeeder"
php artisan db:seed --
class="Database\Seeders\ApplicationPiiPermissionsSeeder"
php artisan db:seed --
class="Database\Seeders\PropertyTaxPiiPermissionsSeeder"
php artisan permission:cache-reset
php artisan optimize:clear
```

The seeders use `updateOrCreate`, so running them again should not create duplicate permission records.

7.7 Inspect legacy plaintext data

Before changing existing data, run the dry-run commands:

```
php artisan pii:encrypt-existing-owners --dry-run
php artisan pii:encrypt-existing-applications --dry-run
php artisan pii:encrypt-existing-property-tax --dry-run
```

These commands report the number of plaintext fields without updating the database.

Review the reported counts before continuing.

7.8 Back up the database

Create a database backup before encrypting existing records, especially in shared development, staging, or production environments.

The encryption backfill updates existing records and should not be run without a recoverable database backup.

7.9 Encrypt existing PII records

After reviewing the dry-run results and confirming that the database backup is available, run:

```
php artisan pii:encrypt-existing-owners
php artisan pii:encrypt-existing-applications
php artisan pii:encrypt-existing-property-tax
```

These commands encrypt only applicable non-null values that are not already encrypted.

7.10 Verify all encrypted records

Run:

```
php artisan pii:verify-owner-encryption
```

```
php artisan pii:verify-application-encryption
php artisan pii:verify-property-tax-encryption
```

Each verification should report:

- zero plaintext fields;
- zero invalid fields;
- successful decryption verification.

Do not disable legacy plaintext compatibility until every verification command passes.

7.11 Disable legacy plaintext compatibility

After all backfills and verification commands pass, change this value in `.env`:

```
PII_ALLOW_LEGACY_PLAINTEXT=false
```

Then run:

```
php artisan optimize:clear
php artisan pii:check-configuration
```

The final configuration check should show that legacy plaintext compatibility is disabled.

7.12 Run the PII test suite

Run:

```
php artisan test --filter=Pii
```

Investigate and resolve any failing test before considering the installation complete.

8. CONFIGURE USERS

The application uses environment variables to create deployment users securely without storing usernames, emails, or passwords in the source code. For now, create only the `Super Admin` account. Other municipality and guest accounts can be configured later when required.

8.1 Configure the super-admin account

Open the project's local `.env` file. Remove or leave out the unused `SEED_MUNICIPALITY_*` and `SEED_GUEST_*` entries.

Keep and complete only:

```
SEED_SUPER_ADMIN_NAME='System Administrator'
SEED_SUPER_ADMIN_USERNAME='superadmin'
SEED_SUPER_ADMIN_EMAIL='<SUPER_ADMIN_EMAIL>'
SEED_SUPER_ADMIN_PASSWORD='<STRONG_UNIQUE_PASSWORD>'
```

Replace the email and password placeholders with secure deployment values. Do not put real credentials in the manual, Git, screenshots, or messages.

Password policy requires at least one capital letter, one number and one special character with minimum 8 characters and max 32 characters.

8.2 Validate and create the user

Open PowerShell in the IMIS project root and run:

```
php artisan config:clear
php artisan seed-users:validate-config
```

Expected result:

```
Seed user configuration is valid.
```

After validation succeeds, run:

```
php artisan db:seed --class="Database\Seeders\UsersTableSeeder" --
force
```

Expected result:

```
INFO  Seeding database.
```

The command creates the configured super-admin account and assigns the `Super Admin` role. If the username already exists, the seeder should not create a duplicate user.

8.3 Verify the user

In pgAdmin Query Tool, run:

```
SELECT id, name, username, email
FROM auth.users
WHERE username = 'superadmin';
```

Confirm that exactly one record is returned, then sign in to IMIS using the configured account.

9. ACCESSING THE WEB APPLICATION

Up to this point, the web application and database has been deployed. To access it, in the terminal, enter the following command within the project file.

php artisan serve

Once the terminal shows success, open the chrome browser and enter the URL:

https://127.0.0.1:8000

The landing page of IMIS opens.

Enter the previously setup user credentials and login.

10. INSTALLING GEOSERVER ON WINDOWS

This chapter installs and configures GeoServer 2.21.0 on Windows for the Base IMIS setup. It uses Java 11, the matching GeoServer 2.21.0 extensions, and the provided Python migration script. Follow the steps in order; do not substitute another GeoServer or extension version while following this guide.

Version note This guide is intentionally written for GeoServer 2.21.0. GeoServer 2.21.x is archived, but it supports Java 8 and Java 11; this guide uses Java 11. The GeoServer extensions installed in this guide must also be exactly version 2.21.0. Do not click a newer "latest" download if you are following this project-specific guide.

Before you start

- Windows administrator access. You may need it to install Java, GeoServer, and copy plugin files into Program Files.
- Download or have these files available: a Windows x64 Temurin/OpenJDK 11 MSI installer (the Java patch version may be newer than the screenshot), GeoServer-2.21.0-winsetup.exe, geoserver-2.21.0-css-plugin.zip, geoserver-2.21.0-querylayer-plugin.zip, geoserver-2.21.0-authkey-plugin.zip, and geoserver-2.21.0-printing-plugin.zip.
- Have the PostgreSQL/PostGIS connection values: host, port, database name, database user, and database password.
- Have the Base IMIS ansible_playbook_server_setup repository downloaded and extracted. Step 9 uses the files/geoserver_workspace/workspace_manifest_tool folder from this repository.
- Have Python 3.10 or newer installed before running the migration script in Step 9.

- Have the Laravel web application folder available so its .env file can be updated after GeoServer is ready.

Installation flow: Java 11 → GeoServer 2.21.0 → GeoServer extensions → database/migration script → web app .env → final verification.

Setup at a glance

Java requirement	Temurin JDK 11, Windows x64
GeoServer release	2.21.0 (Windows installer used in these screenshots)
Web port	8080 by default (choose a free port if necessary)
GeoServer extensions	2.21.0: CSS, QueryLayer, AuthKey, Printing

10.1 Install and verify Java 11

i. Open the Adoptium Temurin download page. Select JDK 11 - LTS. Under Windows, choose the 64-bit MSI download. Figure 1 marks both selections.

   https://adoptium.net/temurin/releases/?version=11

ii. After the MSI file finishes downloading, open it. Follow the installer screens below in order. If Windows displays a User Account Control prompt asking whether to allow the installer to make changes, click Yes.

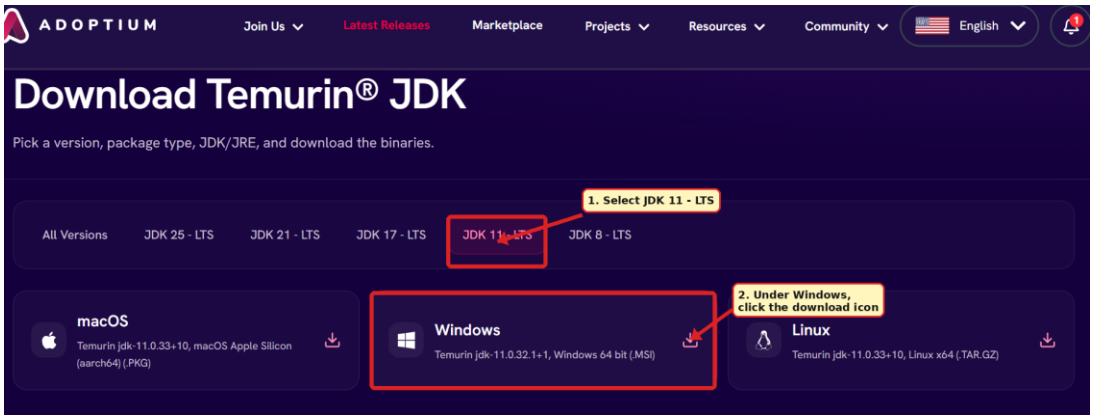


Figure 3. Adoptium download page with JDK 11 selected; choose the Windows installer.

iii. If the Open File - Security Warning window appears, click Run.



Figure 4. Java installer - Security Warning: click Run.

iv. On the Eclipse Temurin JDK Setup Wizard welcome screen, click Next.

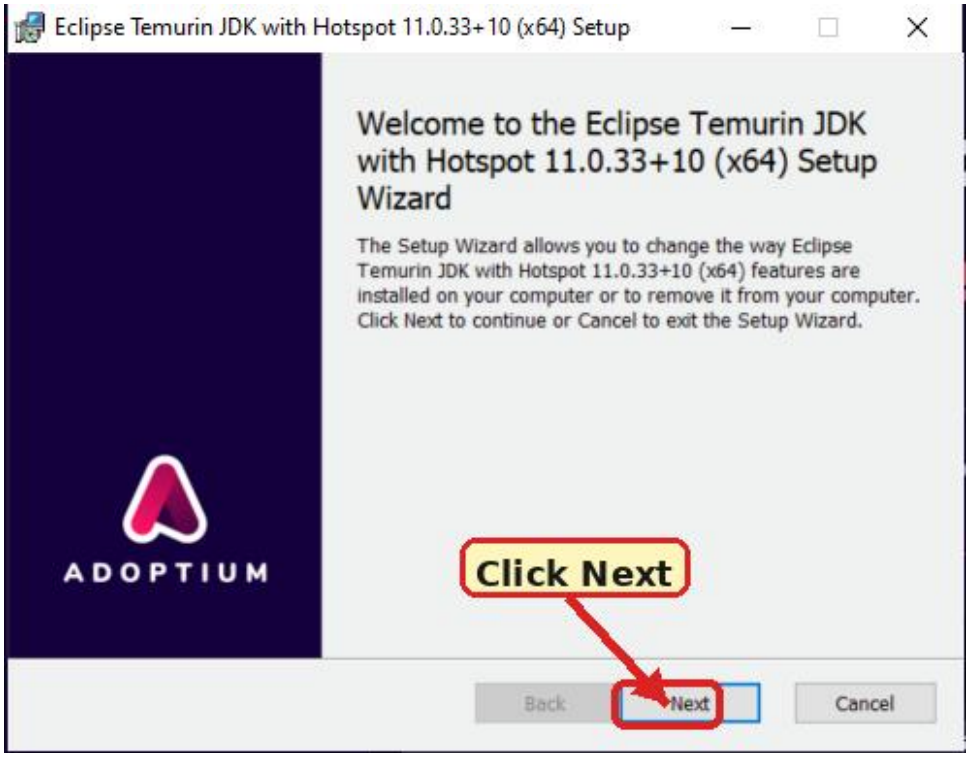


Figure 5. Java installer - Welcome screen: click Next.

v. Read the license screen, tick I accept the terms in the License Agreement, and click Next.

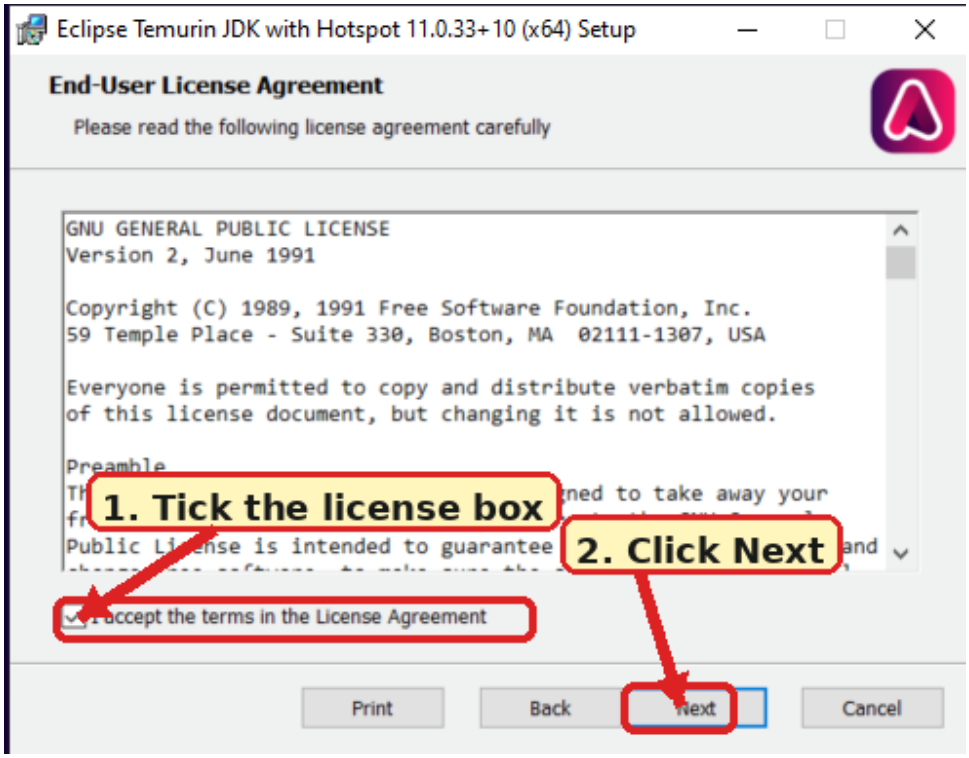


Figure 6. Java installer - License Agreement: tick the checkbox, then click Next.

vi. On Installation Scope, select Install for all users of this machine, then click Next. Administrator permission may be requested.

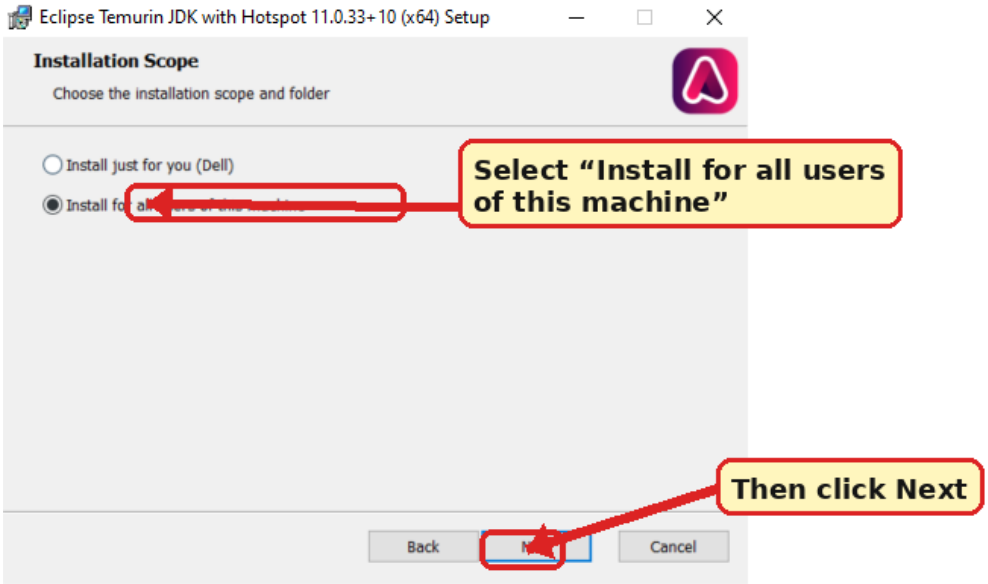


Figure 7. Java installer - Installation Scope: select all users, then click Next.

vii. On Custom Setup, keep Modify PATH variable enabled and leave the remaining options as shown. Click Next. Keeping PATH enabled allows commands such as java -version to work from a terminal.

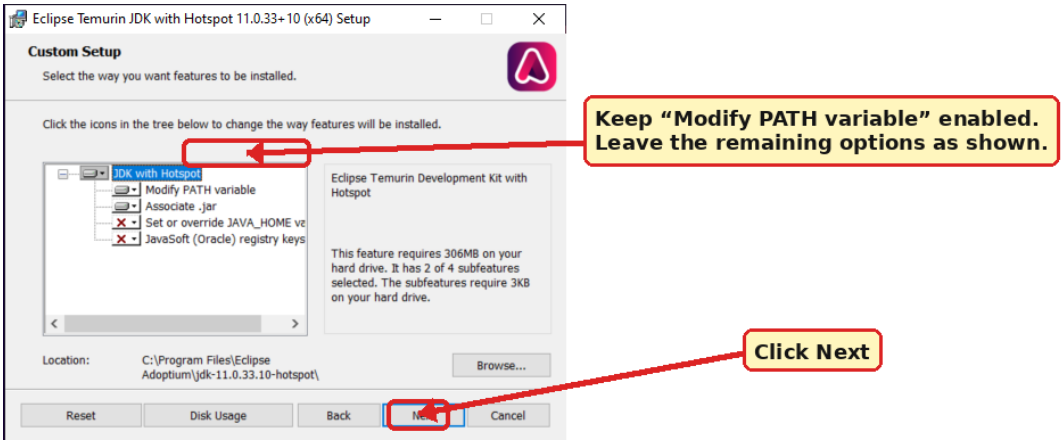


Figure 8. Java installer - Custom Setup: keep PATH enabled, then click Next.

viii. On Ready to install, click Install. Approve any Windows permission prompt if it appears.

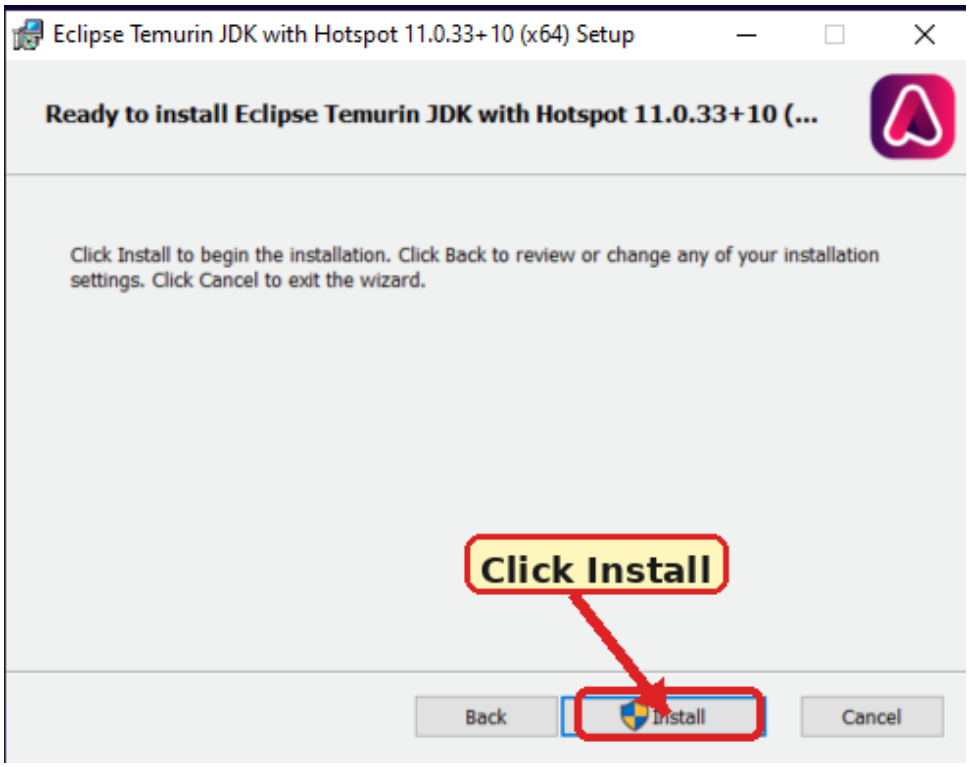


Figure 9. Java installer - Ready to install: click Install.

ix. When the installation completes, click Finish. The JDK is normally installed under C:\Program Files\Eclipse Adoptium\jdk-11.... Keep this folder available because the GeoServer installer may ask you to select it.

x. Open Command Prompt or PowerShell and run the following commands to confirm that Java 11 is installed and available in PATH:

```
java -version
where java
```

Expected result: java -version should show a version beginning with 11. The first path returned by where java should point to a Java 11 installation. If java is not found,

reopen the terminal first; if it is still not found, rerun the installer and confirm that Modify PATH variable is enabled.

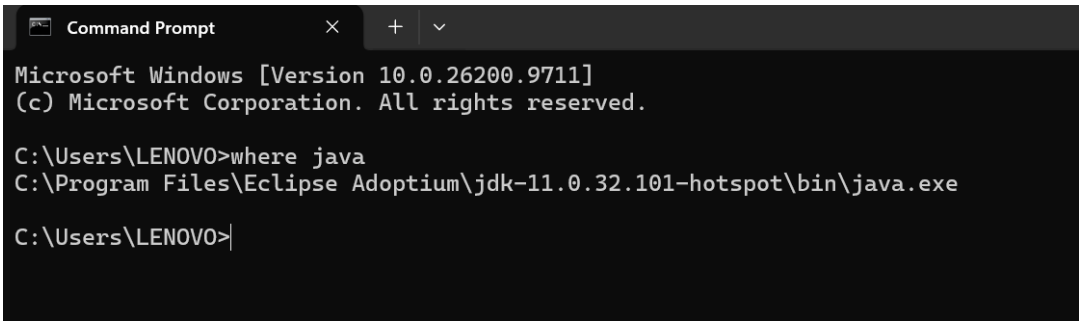


Figure 10. Command Prompt example showing the installed JDK location.

10.2 Download the GeoServer Windows installer

i. Open the GeoServer 2.21.0 release directory in SourceForge:

https://sourceforge.net/projects/geoserver/files/GeoServer/2.21.0/

ii. In the 2.21.0 file list, click GeoServer-2.21.0-winsetup.exe. Use the red box and arrow in Figure 4 to identify the correct file. Do not click the green Download Latest Version button because it can download a newer GeoServer release.

> **Watch out** On SourceForge, the green Download Latest Version button may download a different GeoServer release. Open the 2.21.0 folder and choose the Windows installer explicitly.

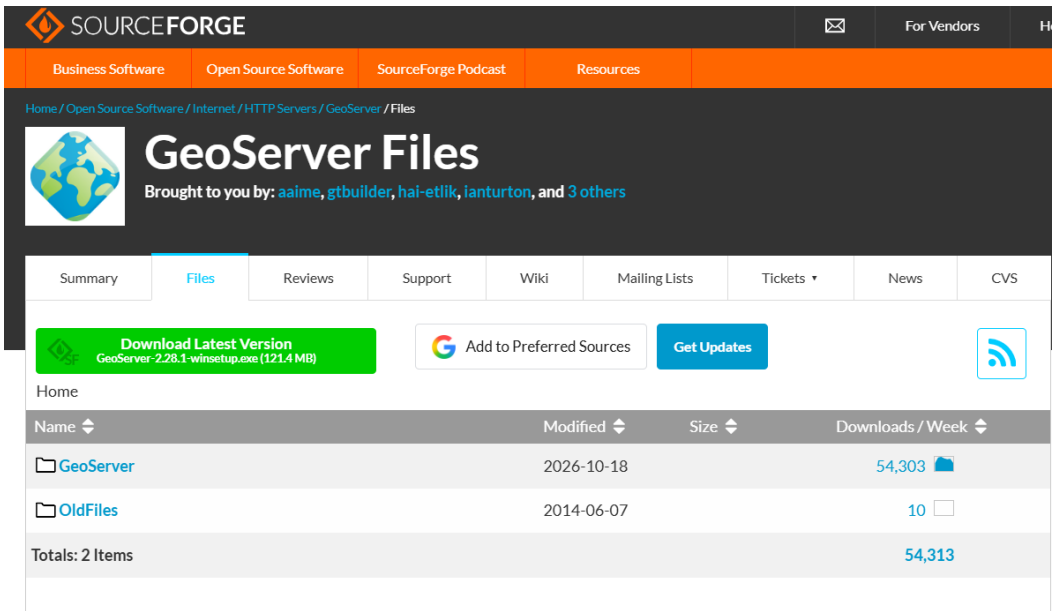


Figure 11. SourceForge GeoServer files area: open the GeoServer folder instead of selecting an unrelated latest release.

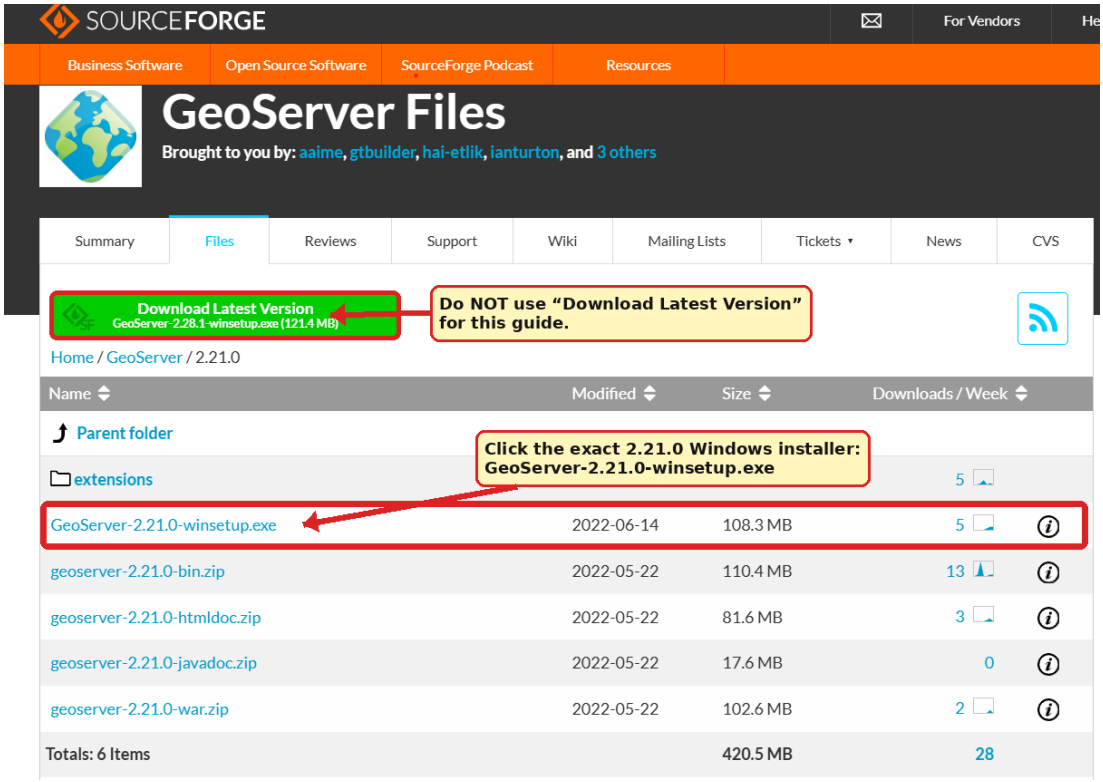


Figure 12. GeoServer 2.21.0 release files: select the Windows setup executable.

10.3 Run the GeoServer setup wizard

Use these choices unless your project administrator has given different values:

- Installation folder: keep the default unless you have a required project path.
- Java Runtime: select the JDK 11 root folder, not the bin folder and not java.exe.
- GeoServer data directory: keep the default for a first installation and note its location.
- Administrator account: create a non-default password and keep it available for Step 9.
- Web server port: use 8080 if free; otherwise choose another unused port and use that same port everywhere later.
- Run mode: choose Install as a service for regular use; choose Run manually only for temporary local testing.

i. Double-click GeoServer-2.21.0-winsetup.exe. Continue through Welcome, License, Installation Folder, and Start Menu options.

ii. At Java Runtime Environment, browse to the JDK 11 installation folder. Select the JDK root folder, not its bin subfolder and not java.exe.

```
Example JDK folder:  C:\Program Files\Eclipse Adoptium\jdk-11.0.32.101-hotspot
```

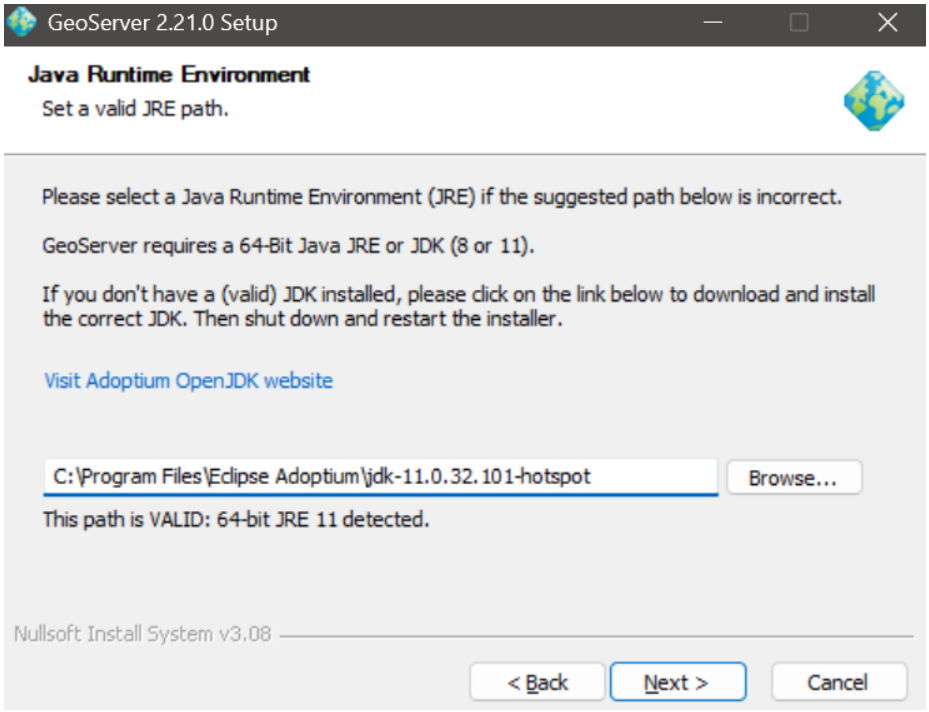


Figure 13. GeoServer installer: a valid JDK 11 folder is accepted as the Java runtime.

iii. Choose a GeoServer data directory. For a first local installation, the default is acceptable; record the chosen location for later backups.

iv. Set the administrator username and a strong, unique password. GeoServer suggests admin / geoserver by default, but you should replace the default password during installation.

v. At Web Server Port, use 8080 if it is free. If another application already uses port 8080, choose an unused port such as 8081 and remember that value.

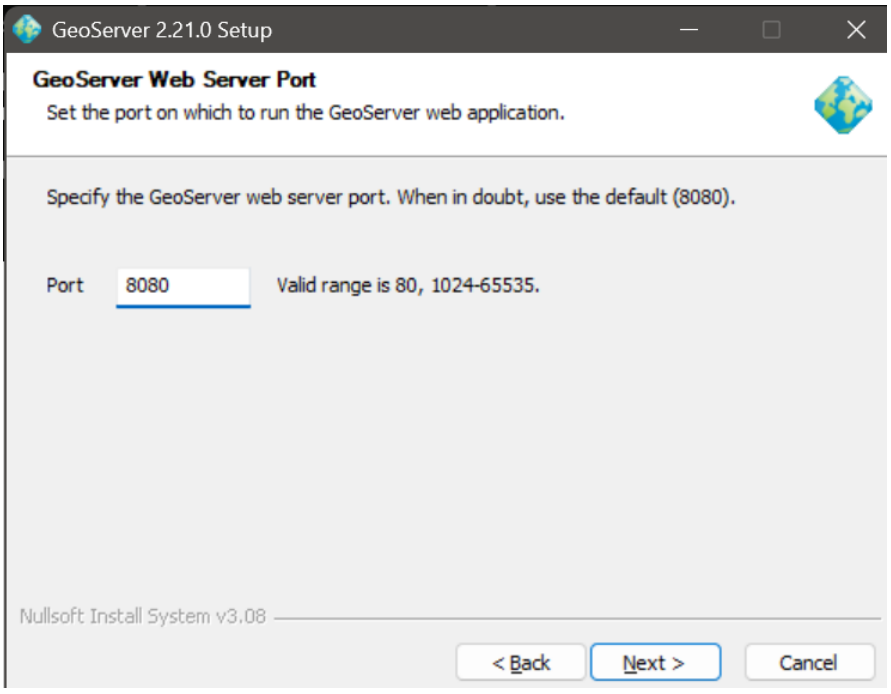


Figure 14. GeoServer installer: default web server port 8080.

vi. Choose Install as a service for regular use on the machine, or Run manually for occasional local testing. Review the settings, click Install, and then Finish.

10.4 Check whether GeoServer is running, then start, stop, or restart it

i. Before starting GeoServer, first check whether it is already running. Open a web browser and go to:

```
http://localhost:8080/geoserver
```

If you selected a different port during installation, replace 8080 with that port (for example, http://localhost:8081/geoserver).

If the GeoServer welcome page opens, GeoServer is already running. You do not need to start it again. Continue to the login step below. If the page does not open, continue to Step

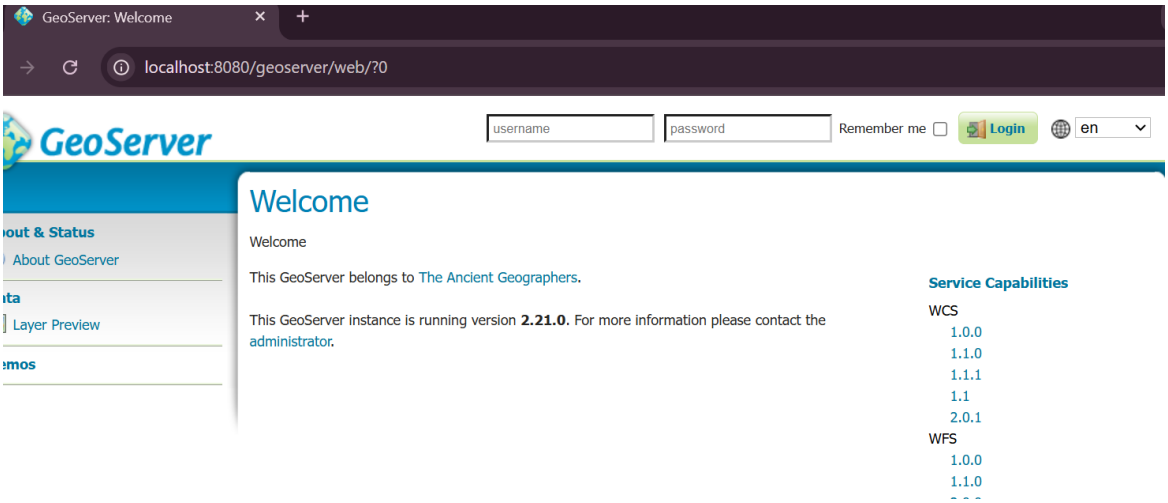


Figure 15. If this GeoServer page opens in the browser, GeoServer is running.

ii. If the page does not open, check the GeoServer service status. Press Windows + R, type services.msc, and press Enter. Find GeoServer in the list and look at the Status column.

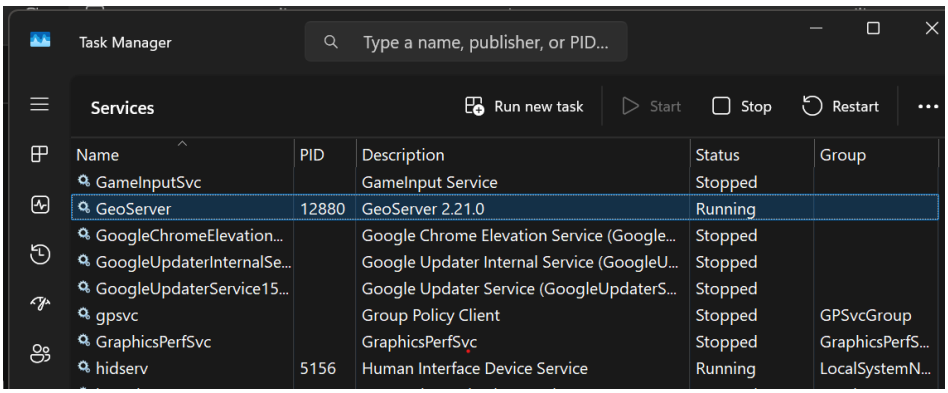


Figure 16. In Windows Services, the Status column shows whether GeoServer is Running. If the Status is blank or stopped, the service is not currently running.

iii. Start, stop, or restart GeoServer from Windows Services as needed. Right-click GeoServer and choose Start, Stop, or Restart. If the Status already shows Running, leave it running unless you specifically need to restart it.

iv. If GeoServer does not appear in Windows Services, or the start/stop shortcuts cannot be found, open File Explorer and go to:

C:\Program Files\GeoServer\bin

Use the batch file that matches how GeoServer is running:

Windows service: double-click startService.bat to start GeoServer and stopService.bat to stop it.
Manual mode: double-click startup.bat to start GeoServer and shutdown.bat to stop it.

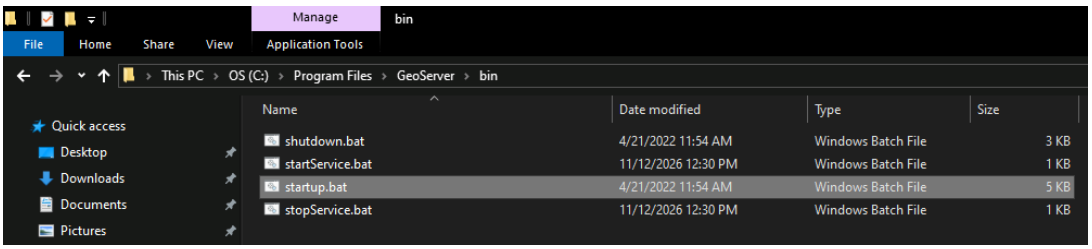


Figure 17. Fallback controls in C:\Program Files\GeoServer\bin. Use startService.bat / stopService.bat for service mode, or startup.bat / shutdown.bat for manual mode.

Important Do not run both sets of files. Use the pair that matches how GeoServer was installed or started.

v. If GeoServer was installed in manual mode and the shortcut is available, open the Windows Start menu, search for Start GeoServer, and open it. Use the corresponding Stop GeoServer shortcut when you need to stop it.

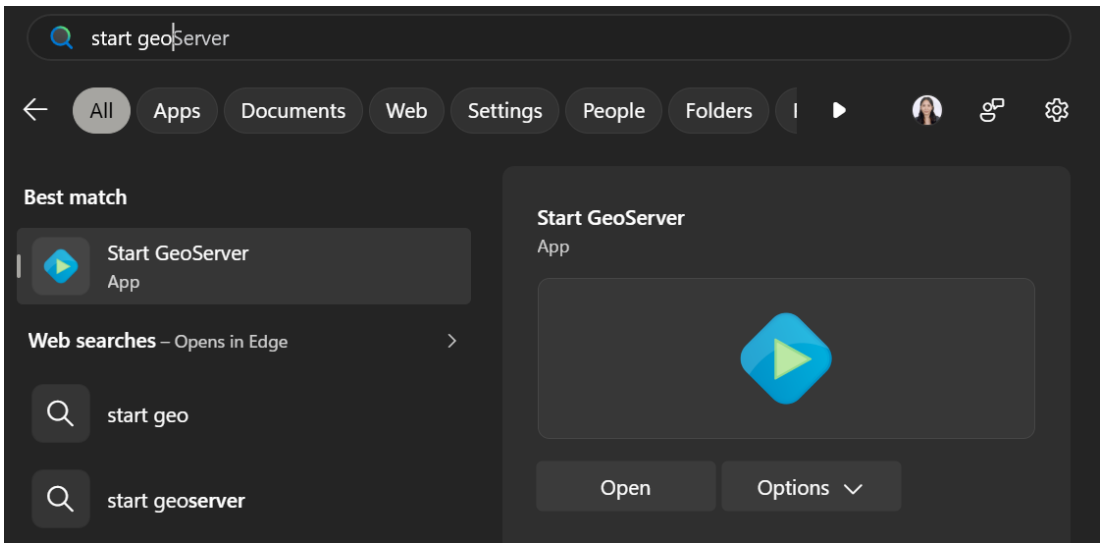


Figure 18. Start GeoServer from the Windows Start menu when using manual mode.

vi. After GeoServer is running, open the browser address again and log in using the administrator credentials chosen during installation.

Expected result The GeoServer welcome page opens and you can sign in successfully. Do not continue with the GeoServer extensions or workspace configuration until this works.

> **Security reminder** The default admin / geoserver combination is only the installer's suggested starting credential. Use the password you set in the wizard; do not leave the default password active.

10.5 Install the GeoServer extensions

Install the GeoServer 2.21.0 extensions used by this Base IMIS setup. The extension version must match the GeoServer version exactly.

i. Open the GeoServer 2.21.0 extensions folder shown below:

https://sourceforge.net/projects/geoserver/files/GeoServer/2.21.0/extensions/

ii. In the extensions list, download all four ZIP files below:

- geoserver-2.21.0-css-plugin.zip
- geoserver-2.21.0-querylayer-plugin.zip
- geoserver-2.21.0-authkey-plugin.zip
- geoserver-2.21.0-printing-plugin.zip

The screenshots below show the exact files to select.

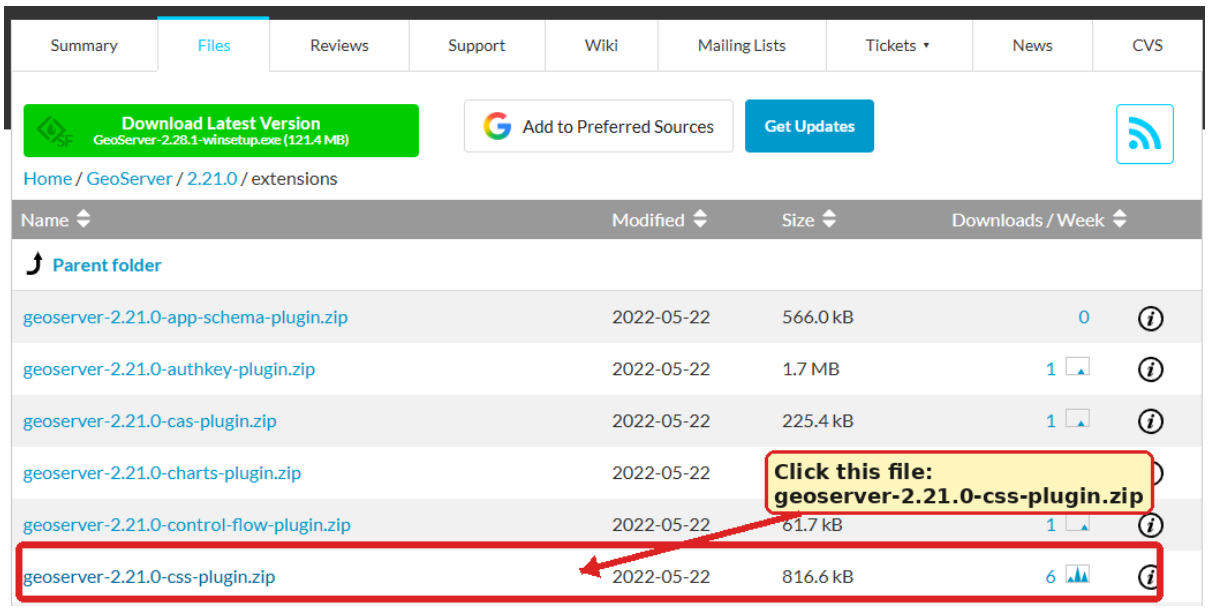


Figure 19. GeoServer 2.21.0 extensions page: download the CSS extension shown above.

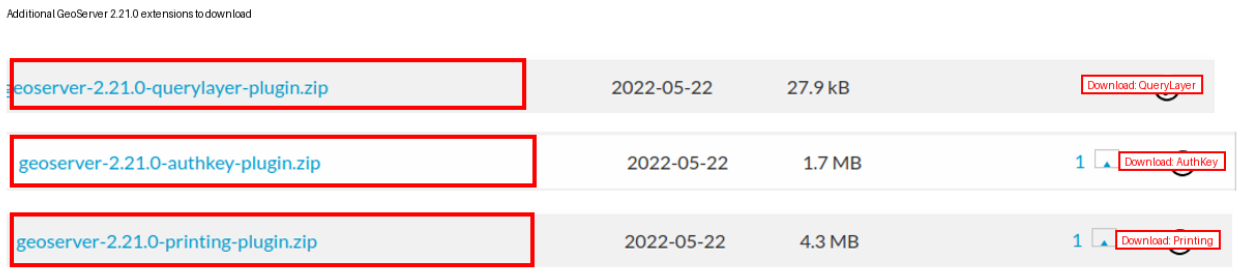


Figure 20. Additional GeoServer 2.21.0 extensions: QueryLayer, AuthKey, and Printing.

iii. Save all four ZIP files to a location you can find easily, such as Downloads. Do not copy files into GeoServer until GeoServer has been stopped in the next step.

iv. Stop GeoServer before changing its extension files. Use the stop method explained in Step 4. Continue only after GeoServer has fully stopped.

v. Extract each of the four extension ZIP files. Open each extracted folder and select the .jar files inside it. Do not copy the ZIP files themselves, README/license files, or enclosing folders into GeoServer.

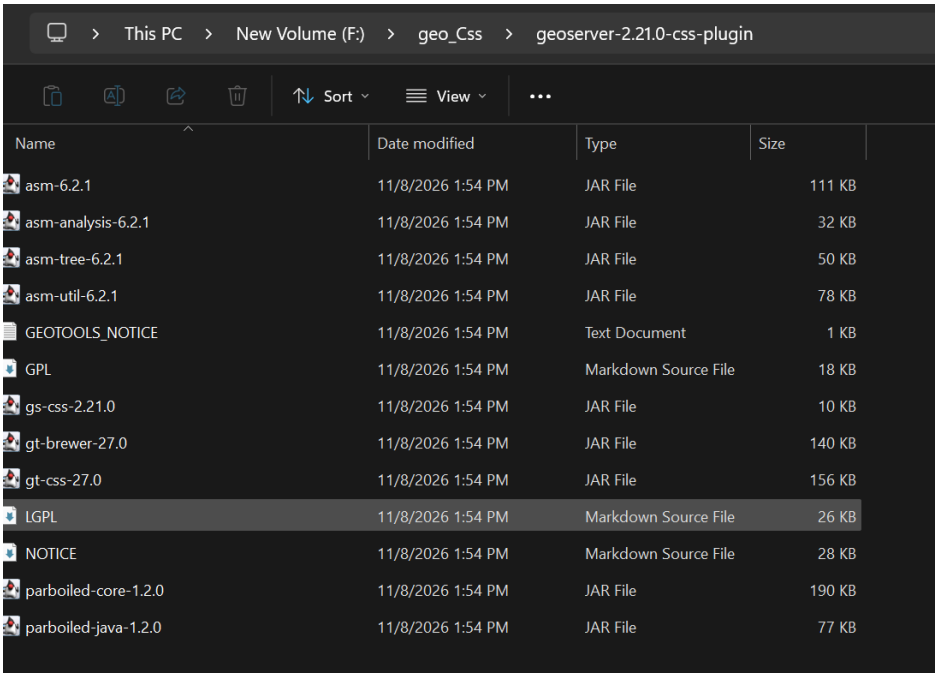


Figure 21. Example: extracted GeoServer 2.21.0 CSS extension. Repeat the same extraction process for QueryLayer, AuthKey, and Printing.

vi. Paste the extracted .jar files from all four extensions directly into the GeoServer library folder below. If GeoServer was installed somewhere else, use the matching WEB-INF\lib folder in that installation.

```
C:\Program Files\GeoServer\webapps\geoserver\WEB-INF\lib
```

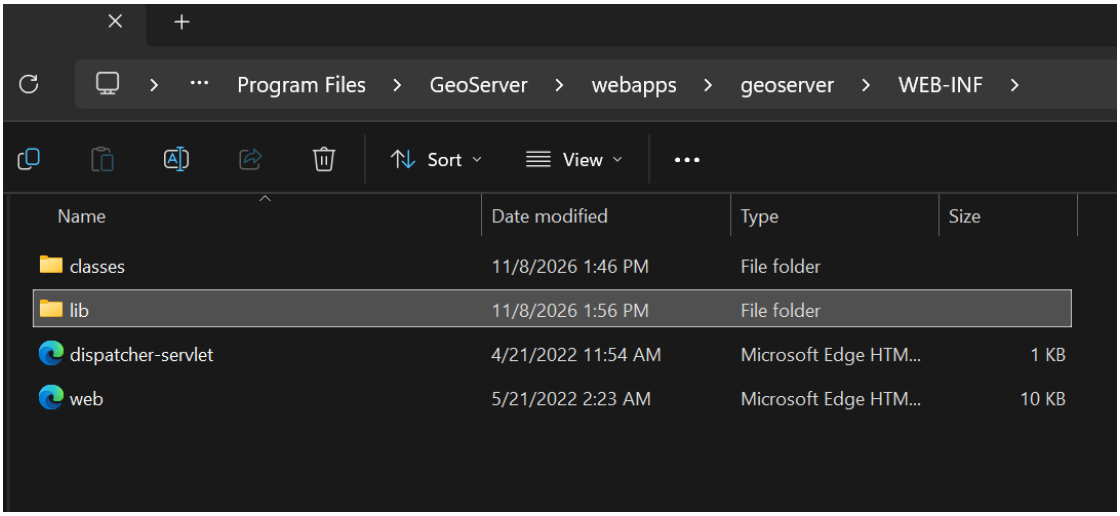


Figure 22. GeoServer WEB-INF folder: open lib and place the plugin JAR files directly inside it.

> Important GeoServer and every extension installed here must use the same version number (2.21.0 in this guide). Do not mix JAR files from different GeoServer releases. Windows may request administrator permission to write to Program Files.

10.6 Restart GeoServer safely

i. After copying the extension files, restart GeoServer using the start/stop/restart method explained in Step 4.

ii. Return to the browser and refresh the GeoServer page. Wait for the application to start before checking the plugin.

> Avoid force-closing Do not end the GeoServer process in Task Manager. Use the Windows Services controls, GeoServer shortcuts, or the batch-file fallback described in Step 4.

10.7 Verify that CSS styling is available

i. Sign in to GeoServer. Look at the left sidebar. Under Data, click Styles, as shown below.

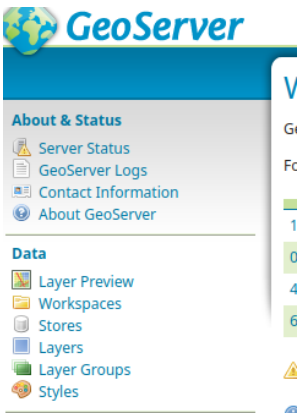


Figure 23. GeoServer left sidebar: under Data, click Styles.

ii. On the Styles page, click Add a new style (or Create a new style).

iii. On the New style page, open the Format dropdown. Confirm that CSS appears alongside the other available style formats, such as SLD.

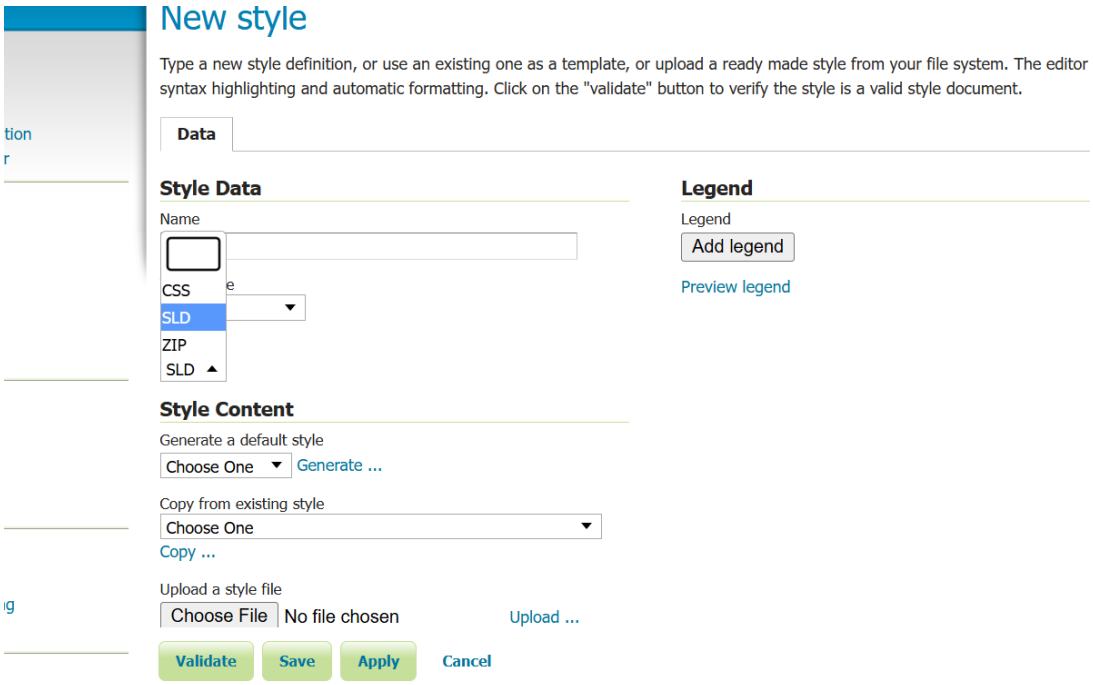


Figure 24. CSS appears in the new-style Format dropdown after the extension has loaded.

Expected result If CSS appears in the Format dropdown, the CSS extension has loaded successfully. You can cancel this test style after checking; you do not need to save it.

10.8 Configure and run the Python workspace migration script

The Python migration tool used in this step is stored in the Base IMIS ansible_playbook_server_setup repository under files/geoserver_workspace/workspace_manifest_tool. Download and extract the complete repository before continuing. Keep the entire workspace_manifest_tool folder together because the importer uses the Python script, manifest JSON, requirements.txt, and icons folder together.

Repository needed for Step 8: https://github.com/base-imis/ansible_playbook_server_setup

Downloading the Repository

i. Download repository from GitHub.

Open the Base IMIS repository using the clickable link above. Click Code, choose Download ZIP, and extract the ZIP to a normal folder on your computer. Inside the extracted repository, navigate to files > geoserver_workspace > workspace_manifest_tool. The screenshot below highlights the workspace_manifest_tool folder location.

Keep the complete workspace_manifest_tool folder together. Do not copy only the Python script. The folder should contain create_workspace_from_manifest.py, the manifest JSON file, requirements.txt, and the icons folder because these files are used together during the GeoServer migration.

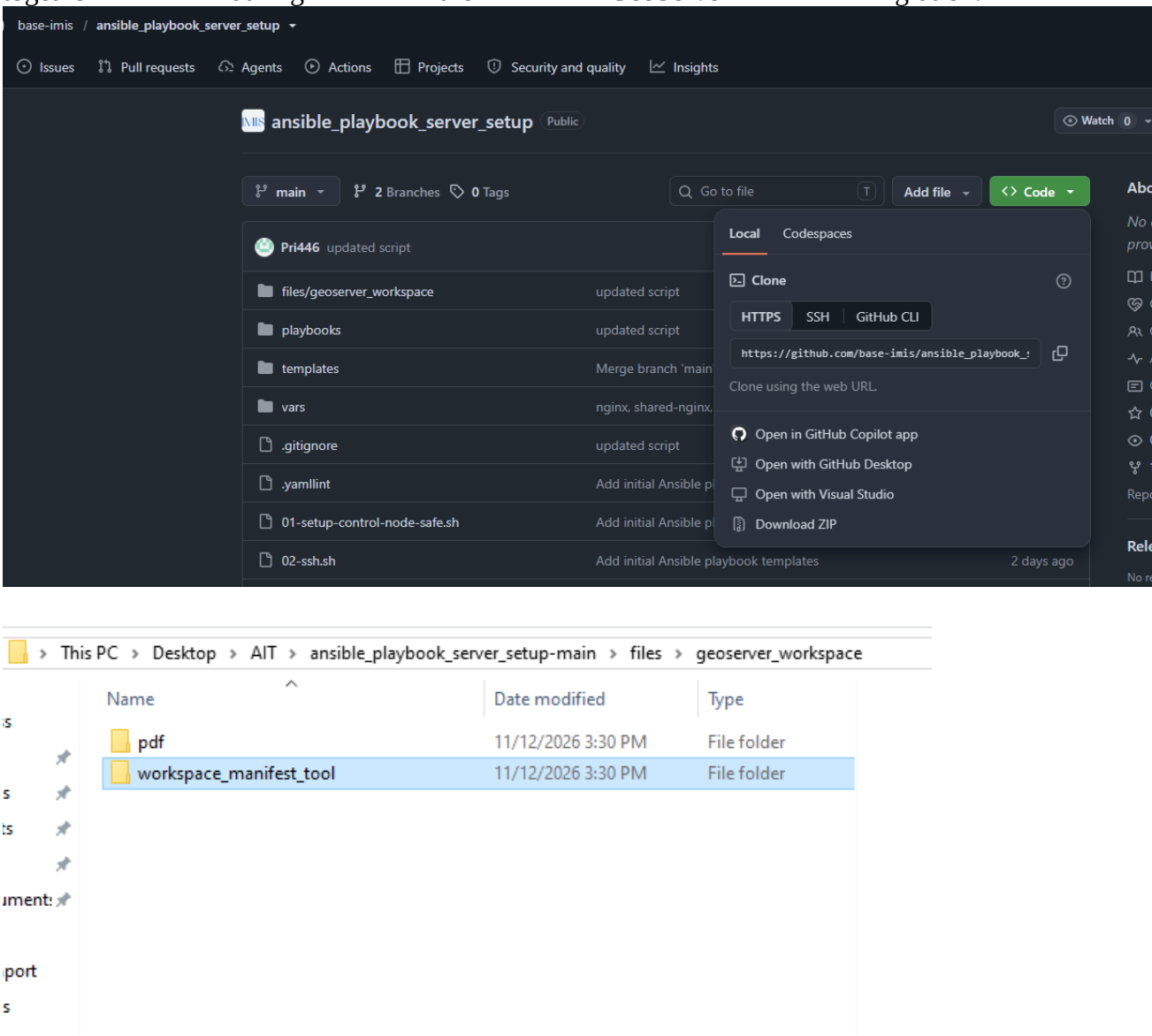


Figure 25. workspace_manifest_tool folder location under ansible_playbook_server_setup/files/geoserver_workspace.

Confirm that create_workspace_from_manifest.py, the manifest JSON file, requirements.txt, and the icons folder are present before continuing.

Before running the script

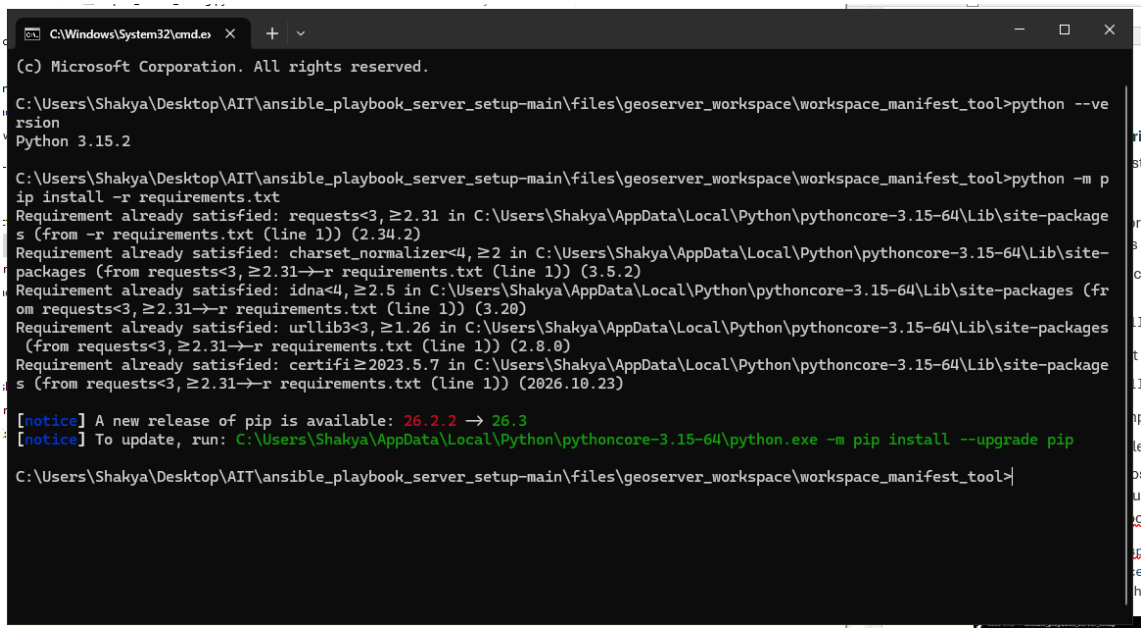
Confirm Python is installed. Open PowerShell and run:

```
python --version
```

Expected: Python 3.10 or newer. If python is not recognized, install Python first and make sure the installer option to add Python to PATH is enabled.

ii. Install the Python package required by the migration script. If the migration folder contains requirements.txt, run:

python -m pip install -r requirements.txt



If requirements.txt is not present, install requests directly:

python -m pip install requests

iii. Run the Python script.

In the terminal, run the command below:

python create_workspace_from_manifest.py

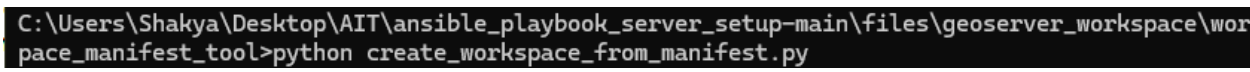


Figure: Executing the python script in the terminal.

If the python command is not recognized but Python is installed, try: py create_workspace_from_manifest.py

The script is interactive for this tutorial. After the command starts, answer the questions shown in terminal. The prompts will ask for the manifest path, destination GeoServer connection details, and the destination PostgreSQL/PostGIS connection details such as host, port, database, user, password, and database type. If a value shown in brackets is already correct, press Enter to accept it; otherwise type the correct value. When a password is requested, the cursor may not move and the characters may not appear—type the password normally and press Enter.

C:\ ansible_playbook_server_setup-main\files\geoserver_workspace\workspace_manifest_tool>python create_workspace_from_manifest.py

Manifest JSON file [C:\Users\Shakya\Desktop\AIT\ansible_playbook_server_setup-main\files\geoserver_workspace\workspace_manifest_tool\imis_base_source_configuration.json]:

Target GeoServer URL [http://localhost/geoserver]: <http://localhost:8083/geoserver>

GeoServer username: admin

GeoServer password: <your_geoServer_Password>

Workspace name: <imis-testing>

Namespace URI (blank derives it from the GeoServer URL): <blank>

Shared database connection (used by all stores)

Database host [postgres14]: <127.0.0.1>

Database port [5432]: <port>

Database name [imis-base]: <database_name>

Database user [base_user]: <database_username>

Database password: <your_dbPassword>

Database type [postgis]: <postgis>

For example:



Do not close terminal while the script is running.

iv. Let the script complete the workspace setup.

The script creates or updates the workspace and PostGIS stores, creates the required layers, uploads local icons, creates the styles, and applies the layer/style configuration from the manifest.

Keep the icons folder beside the script so SVG and PNG symbols used by GeoServer styles can be uploaded correctly. At the end, check the terminal for the Missing resources summary.

Expected result. Near the end, PowerShell should show “Missing resources: none” followed by “Completed manifest-based workspace creation.” If resources are listed as missing, do not continue until those items are corrected.

v. Verify the result in GeoServer before continuing:

[ ] Workspace is available.
[ ] PostGIS stores connect successfully.
[ ] Required layers are present.
[ ] Styles are present and assigned to the correct layers.
[ ] Layer Preview loads the expected map data and icons.

10.9 Configure the project .env file

i. Open the web application folder in Visual Studio Code. In the project root - the same folder that contains artisan and composer.json - open the file named .env.

   Example location:
   <base-imis-web_app>\.env

ii. In the.env file, find the GEOSERVER and GEOSERVER_URL lines. If they are not already present, add them near the other application URL/configuration values. Set them to the values below:

```
#GEOSERVER WORKSPACE
GEOSERVER_WORKSPACE=<workspace_name>


#SERVER GEOSERVER
GEOSERVER_INTERNAL_URL=http://localhost:8080/geoserver/<work
    space_name>
GEOSERVER_AUTH_KEY=<secret_auth_key>
MAPFISH_PRINT_URL=http://localhost:8080/pdf/print.pdf
```

iii. Save the .env file.
iv. Clear the project configuration cache. From the Laravel project directory, run:

   php artisan config:clear

Expected database result: If project can connect to PostgreSQL, the command should display the migration status instead of a database connection error.

v. Open the application page or map that uses GeoServer and confirm that the expected map layers load.

If the database connection fails

Message / symptom	What to check
could not find driver	Confirm the PHP pdo_pgsql extension is enabled.
connection refused	Confirm PostgreSQL is running and DB_HOST / DB_PORT are correct.
password authentication failed	Check DB_USERNAME and DB_PASSWORD.
database does not exist	Check that biren_local exists and DB_DATABASE is spelled correctly.
permission denied	The PostgreSQL user does not have permission to access the required schema/table.

Expected result
Laravel can connect to the biren_local PostgreSQL database, and the application can request the configured local GeoServer workspace and load its map layers.

Quick troubleshooting

Use the table below for the most common setup problems.

Problem	What to check
GeoServer does not start	Verify the selected Java installation is compatible with GeoServer 2.21.0 and check the GeoServer service in services.msc.
localhost:8080/geoserver does not open	Confirm GeoServer is running and use the actual port selected during installation.
CSS is missing from the Format menu	Confirm the CSS plugin exactly matches the GeoServer version, its JAR files are directly in WEB-INF\lib, and GeoServer was restarted.
Access denied when copying JAR files	Stop GeoServer first and use an administrator-authorized File Explorer session. Do not change permissions on unrelated Windows folders.

Problem	What to check
Layer Preview is empty or fails	Check the PostGIS store connection, schema, table permissions, layer SRS, and whether the target table contains data.

10.10 CORS issue - GeoServer images or layers are blocked

Typical browser error
Access to a GeoServer image or layer has been blocked by CORS because no 'Access-Control-Allow-Origin' header is present on the requested resource.

If the browser reports a CORS error while loading GeoServer images or layers, use the steps below to enable the CORS filter in GeoServer's web.xml file.

A typical browser message says that access to a GeoServer image or layer has been blocked by CORS because the 'Access-Control-Allow-Origin' header is not present.

i. **Locate and open the GeoServer web.xml file in a text editor.**

Windows example:
C:\Program Files\GeoServer\webapps\geoserver\WEB-INF\web.xml

ii. Enable Block 1 - the CORS <filter> definition. Find the commented CORS filter block for your server type.

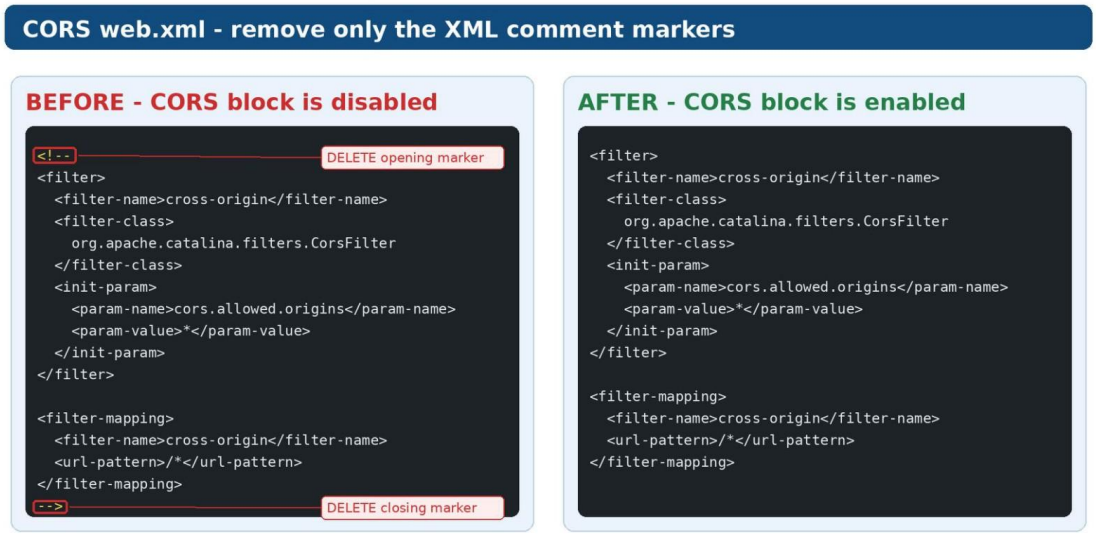
For Jetty Service:

```
<!-- Uncomment following filter to enable CORS in Jetty. Do not forget the second config block further down.
 <filter>
   <filter-name>cross-origin</filter-name>
   <filter-class>org.eclipse.jetty.servlets.CrossOriginFilter</filter-class>
   <init-param>
     <param-name>chainPreflight</param-name>
     <param-value>false</param-value>
   </init-param>
   <init-param>
     <param-name>allowedOrigins</param-name>
     <param-value>*</param-value>
   </init-param>
   <init-param>
     <param-name>allowedMethods</param-name>
     <param-value>GET,POST,PUT,DELETE,HEAD,OPTIONS</param-value>
   </init-param>
   <init-param>
     <param-name>allowedHeaders</param-name>
     <param-value>*</param-value>
   </init-param>
 </filter>
 -->
```

For a Tomcat deployment, use the Tomcat block.

```
<!-- Uncomment following filter to enable CORS in Tomcat. Do not forget the second config block further down.
 <filter>
   <filter-name>cross-origin</filter-name>
   <filter-class>org.apache.catalina.filters.CorsFilter</filter-class>
   <init-param>
     <param-name>cors.allowed.origins</param-name>
     <param-value>*</param-value>
   </init-param>
   <init-param>
     <param-name>cors.allowed.methods</param-name>
     <param-value>GET,POST,PUT,DELETE,HEAD,OPTIONS</param-value>
   </init-param>
   <init-param>
     <param-name>cors.allowed.headers</param-name>
     <param-value>*</param-value>
   </init-param>
 </filter>
 -->
```

iii. Remove the opening <!-- and closing --> around the block as well. After this, both the CORS filter definition and the CORS filter mapping must be active.



iv. Save the file and restart the geoserver so the CORS change takes effect.